Table of Contents

Frontmatter

Frontmatter

Secure all the things with the Spring Authorization Server by Josh Long

ISBN: {isbn} Version 1.0.

without limitation, responsibility for damages resulting from the use of or reliance on this work.

The use of the information and instructions contained in this work is at your own risk.

If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

The Spring Authorization Server

Introducing OAuth

I have been using Yelp [https://www.yelp.com] for a *very* long time.

Since 2007, I'd guess. I was living in Phoenix, Arizona at the time, and I'd started making regular pilgrimages to my hometown of Los Angeles, California. While I was there, I wanted to make the most of the limited time I had in town, so I started looking for tools to help.

Enter: Yelp.

This was the beginning of my journey as a prolific traveler. I love travel. I maintained my Yelp account and even uploaded a profile photo - one that's still there to this day, I think - of me from a trip to Manila, Philippines.

I loved my account. I love Yelp! It connected me with the world. But there's a limit to my love, and a limit to how much I want it connecting with *my* world.

Did you know there was a (mercifully) brief period when Yelp asked users for their Gmail usernames and passwords so it could scan their contacts and help them connect with friends already on Yelp? Yes, that's right. Yelp wanted you to authenticate with your Gmail account on their website so they could log in to your email and *poke around*, basically.

Insanity! Or at least, by today's standards, it is. But back then, there really wasn't an alternative.

The All-or-Nothing Problem

The internet was becoming a more connected place. Companies were beginning to build on the promise of interconnected services, and these kinds of hurdles were common. Yelp wasn't the only site that asked you to enter credentials on its property so it could log in on your behalf somewhere else.

And we all just sort of went with it because, after all, what was the alternative?

This was the all-or-nothing problem. And it led to even more problems.

I trusted Yelp, so I'm sure it would have handled my data with care. But the same couldn't be said of every service that asked. This created a fraught situation where, with every integration and authentication, you were potentially opening yourself up to catastrophic data loss, a loss of privacy, or worse. Yelp would've been fine, but what about some less-than-scrupulous website? What if they went rogue with your credentials? You might not even have had a way to undo the damage if you later changed your mind, because they had your password; they could lock you out of your own account. Also: how would most of these websites have stored your password? Presumably, they'd have to use a two-way hash, encoded using some sort of algorithm and then decoded with the key. But that means that your Google password was one stolen key away from being stored in plaintext! As bad as the AI-driven CVE situation is today, can you imagine if every site you integrated with had access to your *Google password*? You'd have to trust that every one of them doesn't get *pwned*! Sleeping would be *much* more difficult! OK, I'll stop. I know it's all too deeply upsetting to consider. Mercifully, things improved.

Open Authorization for an Open Web

Developers across the SaaS community - particularly at Twitter and Ma.gnolia - realized there had to be a better way for systems and services to federate identity without sharing passwords.

OpenID, the reigning identity protocol of the day, handled *who you are* well enough, but it didn't cleanly cover the problem Twitter and Ma.gnolia kept hitting: third-party apps that wanted to *act on a user's behalf* without ever holding the user's password. By 2007, Chris Messina (yes, the hashtag guy), David Recordon of Six Apart, Twitter's Blaine Cook, and Ma.gnolia's Larry Halff were sketching out something new. Yahoo!, AOL, Flickr, and Google all pitched in engineering effort.

In December 2007, the final draft of OAuth 1.0 was released to the world.

OAuth

OAuth (Open Authorization) is a valet key for the internet. A valet key starts the car and opens the driver's door, but it won't open the glove box or the trunk. OAuth works the same way: it lets a third-party application act on your behalf with a narrowly scoped key, without ever handing over your password.

You have probably used OAuth before. If you've ever clicked "Log in with Facebook" or "Log in with Google" or "Log in with Apple" on a website, you were using OAuth. Instead of creating a new username and password for that site, you're granting it permission to read a small slice of information from your account at the identity provider - usually just enough for the site to put together a profile and know what to call you.

People incorporate OAuth into their applications in all manner of different ways, but the basics are the same. You're using an *OAuth Client* - the application running in your browser, on your phone, or on your desktop - and it tries to access a resource that requires a token.

The OAuth Client redirects you to an *OAuth Authorization Server* (like Okta, Keycloak, Auth0, Active Directory, Clerk, etc.) where you authenticate. It is this OAuth Authorization Server that knows who you are, or can talk to something that does. Once you've authenticated, the OAuth Authorization Server issues a token and sends it back to the OAuth Client. There are several different flows for that handoff, each with its own trade-offs; we'll meet them later.

The OAuth client can now make requests on behalf of the user to APIs (these are called *Resource Servers*) using that OAuth token.

These *Resource Servers* will receive requests and need to validate them. If they receive a request that doesn't have a token, then the request is rejected. If the request has a token, and the token is a stateless JSON Web Token (JWT), then the token is signed and can be validated cryptographically without talking to the Authorization Server for each request. This works because the resource server has obtained the Authorization Server's public cryptographic keys on startup. The resource server validates the token *in-memory* using these public keys. It verifies the signature, the expiration date, and the issuer completely offline.

Otherwise, if the token is stateful (an opaque reference token), then the resource server *must* contact the Authorization Server on every single request.

The tokens contain information about what rights (*claims*) the user has. These tokens support *authorization*.

OpenID Connect (OIDC)

OAuth was a huge leap forward, but it didn't handle every interesting use case. One of the first things people did once they had a valid access token was use it to ask the provider's API *who is this user?* - their email, their name, a profile picture - and then sign them into their own application based on the answer. The implication was that providers like Google or Facebook had already done the work of confirming the email belonged to the person (and sometimes even checked government-issued ID along the way), so an application could trust that a returned email really *was* that person's email. If Google and Facebook both returned the same email, it really was the same person across two different ecosystems. This pattern became the basis of the early *Sign In with <X>* capabilities all around the internet, and let services focus on building interesting things instead of solving identity over and over again.

People were using OAuth to do *authentication*, and it caught on fast. The trouble was that every provider exposed those identity details in its own bespoke way, and it became pretty self-evident after OAuth 2.0 that there was a gap.

Enter OpenID Connect (OIDC).

OIDC launched in 2014 as a thin, standardized identity layer on top of OAuth 2.0. Its creation was a massive collaborative effort involving engineers from Google, Microsoft, Salesforce, Ping Identity, and Yahoo, with Nat Sakimura, John Bradley, Michael B. Jones, Chuck Mortimore, and Breno de Medeiros among the key contributors.

What OAuth Gives You

OAuth introduced three things you didn't have when applications were just trading passwords around.

First, **tokens instead of credentials**. Even if an attacker steals a token, they only have access for a limited time and only to the data the token was issued for.

Second, **scopes**. The user (or the developer on their behalf) declares up front what an application is allowed to do - read your calendar, post on your behalf, see your profile - and the token carries that entitlement. The valet key gets the car moving without unlocking the trunk.

Third, **revocation**. The user can cut off an application's access at any time without changing their password and without affecting any of their other connected apps.

There are many scenarios where this matters:

- Third-party applications and services: applications that want to access services like Google Drive, Twitter feeds, or Facebook posts.
- Mobile applications: smartphone applications that access web services but don't want to store passwords.

- Content aggregation: an app that pulls data from multiple sources (e.g., various email accounts).
- Federation: if you need to maintain secure integrations with a number of OAuth platforms, it's possible to use an OAuth IDP like Spring Authorization Server to act as a facade.

Evolution of OAuth

OAuth has evolved over the years to address shortcomings and to better meet the requirements of developers and organizations. There are two main versions of OAuth:

OAuth 1.0: The original specification, finalized in December 2007 and later refined as RFC 5849 in 2010. It was complex, required clients to sign every request cryptographically, and was painful to implement correctly. OAuth 1.0a tightened a few security gaps, but we'll leave both behind.

OAuth 2.0: Standardized as RFC 6749 in 2012 as a successor, it simplifies the protocol and separates the roles of obtaining credentials from using them. It requires TLS for the authorization and token endpoints - the whole protocol assumes you're operating over HTTPS - which is a large part of what lets it drop OAuth 1.0's request-signing machinery.

OAuth vs. SAML

You'll often see OAuth contrasted with SAML (Security Assertion Markup Language), and you'll often see the contrast drawn as "SAML is for authentication, OAuth is for authorization." That's a useful starting point but it overstates the case - as you just saw, OAuth was being used for authentication well before OIDC formalized it.

The more practical differences:

	OAuth 2.0 / OIDC	**SAML 2.0**
Era and ecosystem	2012 onward; mobile, SPAs, APIs	2005 onward; enterprise SSO
Wire format	JSON, with tokens that may be opaque or JWTs	XML assertions, signed and sometimes encrypted
Transport	HTTP redirects and JSON over HTTPS	HTTP redirects, form POSTs, and SOAP bindings
Typical use	App-to-API delegation; "Sign in with…"	Enterprise SSO into web apps from a central IdP

Both are still actively deployed. SAML is deeply entrenched in enterprise SSO; OAuth 2.0 and OIDC dominate almost everything newer.

Towards a Passwordless Future

One of OAuth's quiet superpowers is that it centralizes authentication. Authentication is a hard problem, and there are a million things to get right for it to even begin to be considered secure. Okta, Google, Meta, GitHub, and the Spring team have all committed serious engineering effort to getting it right so that you don't have to. Done well, a good OAuth integration can dramatically

reduce the number of passwords you need to maintain to live on the internet.

In this book, we're going to focus on the Spring Authorization Server. Once it's at the heart of your SSO solution, identity for your organization collapses down to a single credential.

But what about *no* credentials? Or better, more secure alternative credentials?

That's where things are heading. Large providers like Apple, Google, and Microsoft are pushing *passwordless* logins tied to device biometrics, built on WebAuthn - and, increasingly, on passkeys, which are discoverable WebAuthn credentials synced across a vendor's ecosystem so you can sign in from any of your devices.

Spring Authorization Server can be extended to participate in this future too, and by the end of the book you'll do exactly that.

Introducing the Spring Authorization Server

In this book we're going to look at how to use Spring's OAuth stack to *secure all the things*. At the center of this discussion, necessarily, must be an OAuth authorization server. To be fair, you could use *any* valid, modern OAuth authorization server to do most of the things we'll discuss in this book. OAuth is an open protocol, after all.

Why Spring Authorization Server

Technically, the Spring Authorization Server is an OAuth and OIDC compliant solution just like any other. But as is so often the case, the decision rests on intangibles that go well beyond just the technical merit of the project.

Really, there are *two* questions before us: why should existing applications move to the Spring Authorization Server, and why should new projects start with it?

It's 2026 as I write this, and if you've got software in production, you probably have an OAuth authorization server deployed somewhere. We are spoilt for choice these days!

The space is crowded. On the commercial side you've got Okta (and the Auth0 product they acquired), Microsoft Entra ID, Amazon Cognito, Google's Identity Platform, and Ping Identity (which now includes ForgeRock), alongside a newer cohort of developer-first SaaS shops like Clerk, WorkOS, Stytch, Frontegg, and Descope. On the self-hosted side, Keycloak is the eight-hundred-pound gorilla, with Ory, Authentik, Zitadel, Dex, and Apereo CAS each holding meaningful turf. There are dozens more.

Most of them would work just fine in place of the Spring Authorization Server. I love the Spring Authorization Server precisely because it works just fine in place of most of *them*, and it works in a way that's natural for anyone already living in the Spring ecosystem.

Let's look at the technical and legal facets that drive that choice.

Technical

The pitch is simple: it's a Spring Boot starter.

If you've ever deployed a Spring Boot application, you already know how to deploy the Spring Authorization Server. The same configuration model, the same Actuator endpoints, the same Micrometer metrics, the same container image story, the same CI/CD pipeline. Your team's existing muscle memory transfers over.

Owning the IDP unlocks a few things that are awkward, expensive, or simply not possible with a SaaS:

- **Your data stays where you put it.** Users persist into the same database your application already uses - the same Postgres instance, the same backup story, the same migration tooling, no data leaving your network. That matters in regulated industries, and it matters anywhere a vendor's data-residency story doesn't line up with yours.

- **Custom claims from your business systems.** You can mint tokens that carry the exact claims your services need - tenant ID, role hierarchy, feature flags - read straight from your domain model, without paying a SaaS for a "rules engine" and writing JavaScript inside a vendor's UI.
- **Federation on your terms.** Hook up to your existing LDAP, your Active Directory, your social providers, your customer's SAML IDP - and present a single, consistent OAuth surface to your application code.
- **The same scaling story as the rest of your fleet.** Horizontal scaling, blue-green deploys, traffic shaping, observability - all the things you already do with Spring Boot apply to your auth layer.

When something doesn't fit, you reach for the same autoconfiguration override pattern you use everywhere else in Spring Boot - declare a bean of the type you want to replace, and Spring Boot backs off its default. There's no support ticket, no quarterly contract renegotiation, no rate limit on your own data.

Legal

Quick disclaimer, then the substance: *I am not a lawyer*, I am not your security consultant, and nothing in this book is legal or compliance advice. I work on the Spring team and I'm here to teach you the technology. I'll point at good practices where I can, but I can't cover every jurisdiction, every industry regulation, and every threat model, so I'm not going to try.

Here's why that matters. User management is a minefield, and when something goes wrong - and eventually, in some form, it will - regulators will ask whether you did everything reasonable to prevent it. Could the Spring Authorization Server be configured and operated in a way that complies with HIPAA, PCI-DSS, GDPR, SOC 2, or whatever else applies to you? Almost certainly yes. Will the out-of-the-box defaults get you there? No. Nobody's defaults get you there - not mine, not anyone else's. That isn't the job of an OAuth server.

To actually pass an audit, you need operational things like:

- backups
- multi-factor authentication
- HSMs (or a KMS) for signing keys
- comprehensive monitoring and alerting
- a patching cadence, and the discipline to keep to it
- a documented incident response program
- (and a lot more besides)

The Spring Authorization Server can participate in all of that, and I'll point at the hooks where it does. The operational discipline around it is on you.

And before you assume a managed SaaS makes this go away: it doesn't. Providers like Okta supply genuinely useful compliance artifacts - SOC 2 reports, ISO 27001 certifications, audit logs, security whitepapers. But regulators still hold *your* organization accountable for how you configure the service, who has access to it, your business processes, your regulatory obligations, and your overall compliance program. The vendor's compliance posture is a useful input. It is not a substitute for

your own.

The Spring Security OAuth Ship of Theseus

The Spring Authorization Server is one of my favorite new projects. It's a full-blown OAuth identity provider (IDP), distributed as Spring Boot autoconfiguration.

But getting to this oh-so-approachable place took more than a decade! It started way back in 2011, when the Spring team volunteered to take on, and evolve, an awesome third-party open-source community project. We wanted to support the project and use it for some of the use cases we had in the popular open-source cloud platform, Cloud Foundry, where it eventually became the UAA [https://github.com/cloudfoundry/uaa] component.

This project had three interesting pieces: *OAuth Resource Server* support, *OAuth Client* support, and *OAuth Authorization Server*. We took it on and even had legends like the great Dr. Dave Syer (cofounder of Spring Boot and Spring Cloud, for a start) work on it. The project worked, but by 2015 it was starting to get long in the tooth. It was built before Spring Boot. It didn't support OIDC and other improvements in the OAuth space. It didn't work in a reactive world. Generally, it needed an overhaul.

So we embarked upon a project to rebuild and reintroduce OAuth support in Spring Security. This time, we said, the OAuth support would live as part of Spring Security itself, not a separate project. The first piece to debut, in the Spring Security 5.0 and Spring Boot 2.0 generation, was the OAuth Client and OIDC support. Then we introduced resource server support. And we said that we didn't feel the need to build an OAuth Authorization Server, as the market had many good choices already.

In a GitHub issue on `spring-security` wherein we declined to build this, the community told us we were wrong. Loudly [https://github.com/spring-projects/spring-security/issues/6320#issuecomment-614218694]. This issue was one of the most vibrant in the Spring ecosystem's history! Clearly, there was demand. And so in 2020, work began in earnest, eventually landing as a fully supported, easily autoconfigured project in Spring Boot that you can use today.

The Spring Authorization Server originally lived as a separate project, with its own lifecycles and release cadences and so on, but once it stabilized, it was moved under Spring Security with the Spring Security 7 release in late 2025.

The Spring Security OAuth project is dead. Long live Spring Security's OAuth support!

One Less `users` Microservice For All

Having a real IDP at the center of your architecture is *liberating*.

The obvious win is consolidation. Every team that would otherwise have built its own `users` table, its own password reset flow, its own session story, and its own permission system can stop. There's one identity surface for the whole organization: one place to onboard a new employee, one place to revoke them when they leave, one place to add SSO into a customer's IDP. You'll have one less bespoke token-management system, too.

The less obvious win is isolation. Password hashing is expensive *on purpose* - bcrypt, scrypt, and

argon2 are designed to burn CPU so attackers can't brute-force them. In a typical monolith, that CPU is competing with every business-logic request you're serving. Move authentication into a dedicated authorization server and the expensive work happens on a node you can scale, monitor, and harden independently of everything else.

And because it's Spring, it composes with the rest of Spring Security, Spring Boot autoconfiguration, and the broader ecosystem - the hooks and customization points are there at every step.

If you knew the agony I've been subjected to learning and wielding OAuth over the years, you'd understand what it means when I say I love this project. It's the first OAuth experience I've had that doesn't feel like work.

And, dear reader, I want you to love OAuth too. The trick is to *stop having to think about OAuth* - to set it up well once, and get back to building the thing you actually meant to build. That's what the rest of this book is about.

So let's take that journey to production together, with the Spring Authorization Server at our backs.

Implementing the Spring Authorization Server

Go to the Spring Initializr (start.spring.io) [https://start.spring.io], specify a group ID and an artifact ID (I chose `bootiful` : `authorization-server`) and add `OAuth2 Authorization Server` as a dependency. I'd add `GraalVM Native Support` for good measure, but you do you. Open the downloaded project in your IDE. I'm using IntelliJ IDEA, but again, you do you. I ran the following command from the root of the newly unzipped archive: `idea build.gradle`.

You've got a new Spring Boot Authorization Server. We need to specify two things: **users**, and **clients**.

Users

Users are pretty straight forward, right? A user is the sum of the username, password, and associated information attached to the system's notion of *identity*. The beating heart of our system. The *tenant* in "multitenant."

There are a lot of ways to get this done. The easiest might be to just have one user, the *default* user, which you can describe using Spring Boot's associated properties, like this:

```
spring.security.user.name=jlong
spring.security.user.password=password
spring.security.user.roles=admin,users.write,users.read,openid
```

This gets us off the ground, but as soon as you want two or more users, you'll need to specify them a different way. The easiest is probably to define a bean of type `InMemoryUserDetailsManager`, like this.

```
package bootiful.authorizationserver;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;

@Configuration
class UserDetailsConfiguration {

    @Bean
    UserDetailsService inMemoryUserDetailsManager() {
        var userBuilder = User.withDefaultPasswordEncoder();
        return new InMemoryUserDetailsManager(

userBuilder.roles("USER").username("josh@joshlong.com").password("pw").build(),
                userBuilder.roles("USER",
```

```
"ADMIN").username("rob@spring.security").password("p@ssw0rd").build()
        );
    }

}
```

Thus configured we've got two users:

- `josh@joshlong.com` with password `pw` and roles `USER`
- `rwinch@spring.security` with password `p@ssw0rd` and roles `USER` and `ADMIN`

This implementation is fine for development as it's all in-memory. In a production system, you'll probably want something more durable. We'll look at those possibilities in a bit. For now, let's talk about OAuth clients!

OAuth Clients

An OAuth client defines how a program or process interacts with an OAuth Authorization Server (like Spring Authorization Server). Clients correspond more or less to the programs that would like to be allowed to act on behalf of users but need their permission - their *access tokens* - to do so.

I have tried to conceive of a clear illustration of clients in a vacuum, but it's not easy. so let's examine a real life example: you stumble upon some website, say Yelp [https://www.yelp.com/], a website that lets you contribute and read reviews about locations - restaurants, businesses, tourist spots, etc. You want to login to see your history. You *could* create a new account there, going through the whole signup flow and entering redundant information, but this information could soon become stale. Worse: you've probably already entered this somewhere else on the internet and done a better job of maintaining it there, too. Maybe you change house or email address, or whatever, and you've forgotten to go back to the site and change your information. Yelp know this, so they offer another path forward: `Continue with Google` and `Continue with Apple`.

[yelp signup] | *./images/yelp-signup.png*

Click the button and another window on Google or Apple's sites pop up.

[google signin] | *images/google-signin.png*

You know what to do here: you're in familiar territory. It's google.com! You know Google. And if you've ever set up an account and stayed logged in while navigating the web, Google *almost certainly* knows you too! You've got an account here, you maintain that account, and you like that account. You use it for your daily email, after all. You've even got that reassuring little padlock icon in the browser's location bar giving you the warm-n-fuzzies about this site's authenticity: it is who it claims to be. So you enter your information, login, and you do whatever mutli-factor auth things Google wants you to do. You might get a prompt on another device, you might use a passkey, etc. Let's suppose we get the prompt on another device.

[google mfa] | *images/google-mfa.png*

This shows up as a prompt on a completely different device, an iPad.

[google is it you] | *images/google-is-it-you.png*

You've approved of the login, so that Google knows it really is you logging in, and now it's got to make sure you realize you're handing over some of the data associated with your identity to this new website, Yelp.com, so it throws up a consent form.

[google consent screen] | *images/google-consent-screen.png*

You click `Confirm` and then are finally logged in, with your Google identity, on Yelp.com

[yelp logged in with google part 2] | *images/yelp-logged-in-with-google-part-2.png*

At the end of this exchange, Google.com transmitted a *token* to the application running at Yelp.com. Armed with this, the application running at Yelp.com can now transmit requests to the Google.com APIs, asking it questions about you, like your email. It might also be able to read your Google calendar events, location data, etc. What precisely the application at Yelp.com has access to is a function of the *scopes* requested by the client. The application at Yelp.com stores the token and uses it to interact with Google on your behalf. Occasionally, Google.com will expire the token. Tokens, like milk, go stale! No worries: the application at Yelp.com was also given a *refresh* token it can use to refresh the access token and get a new one.

You're glad you signed up at Yelp.com, but look at the time! It's noon, the sun's out, and the kid wants to go play mini golf at the place you just found on Yelp.com. Gotta go!

Time passes, and you return to Yelp.com a week later. By this point, Yelp.com's expired your HTTP session, and you're logged out. No problem. Click the `Continue with Google` button again, and this time you'll just be dumped into Yelp.com, fully authenticated. Both Google and Yelp remember who you are, and so there's no ceremony this time. You got fast-path'd into an authenticated HTTP session on Yelp.com. Thus: OAuth is invaluable both for establishing a new account and for subsequently logging into it. You may have changed your home address on Google.com in the meantime, and now Yelp.com can see the new address information and offer you updated recommendations, too. So Yelp.com is kept up to date, and all you had to do was keep Google.com up to date.

From the perspective of Google, Yelp.com is an OAuth client. All the particulars of how you went through that authentication flow - whether you needed to be redirected to Google.com, whether you should be shown a consent form, and what data Yelp.com was allowed to read from the Google.com API once it had a token stemming from this authentication flow, was governed by how the developers at Yelp.com registered their client with Google.

Clients must stipulate a client ID, and a client secret. The client ID and client secret are transmitted in the request initiating the authentication flow, signaling to Google that Yelp.com is making this request. Clients also stipulate what *scopes* they want. A scope is OAuth's version of rights, permissions, authorities, or claims. They're (basically) arbitrary strings that mean something to Google.com's API about what you may and may not access.

There is one scope, `openid`, which is part of the OIDC specification and allows an OAuth client to authenticate users in a standard way. Specifically, it has two effects: * an ID token that contains a JWT that client applications can consume to understand who the user is. * it gives the client access to the `UserInfo` endpoint. This endpoint provides additional information about a given user.

This is a sort of special case; Yelp.com may not want to read Google Calendar data, or read your email, or draft a spreadsheet for you in Google Drive. The scopes to support those Google-y things would necessarily be unique to Google's APIs. But signing a user into a site is a common enough thing and one that can be implemented usefully across all sorts of OAuth providers, so there's a specification called OpenID Connect (OIDC), that builds on top of OAuth 2.0, prescribing standard scopes and , importantly, standard APIs by which a client may look up information associated with a user. Yelp.com might only just need enough information from Google.com to fill out a signup form for us: name, email, etc. In this way the Yelp.com client could even reuse the same code across other OIDC compliant providers, changing only the client ID and client secret and the issuer URI (the API's root URL). Neat-o!

So, if you built a backoffice process, you'd register a client for that backoffice process. If you built a new web application that you intend to support automatic sign-in with OAuth, you'd register a new client for that web application.

The simplest way to register clients in the Spring Authorization Server is to use properties in the `application.properties` or `application.yaml` file, like in this `application.yaml` example:

```
spring:
  security:
    oauth2:
      authorizationserver:
        client:
          crm-client:
            require-authorization-consent: true
            registration:
              client-id: crm
①
              client-secret:
"{bcrypt}$2a$10$m7dGi0viwVH63EjwZc6UdeUQxPuiVEEdFbZFI9nMxHAASTOIDlaVO"
②
              authorization-grant-types: client_credentials, authorization_code,
refresh_token
③
              redirect-uris: http://127.0.0.1:8082/login/oauth2/code/spring
④
              scopes: user.read,user.write,openid
⑤
              client-authentication-methods: client_secret_basic
```

① you can use the `spring` [https://docs.spring.io/spring-boot/docs/current/reference/html/cli.html] CLI to encode a password for the client secret: `spring encodepassword BLAH`, where `BLAH` is the string you want to encode. In our case, the client ID is `crm` and the client secret is `crm`. (Again, I *know* it's a terrible password. Don't `@` me!). NB: For complex strings like this, YAML parsing rules can be problematic, so I tend to wrap these things in quotation marks.

② `authorization-grant-types` refers to the use case - web application, mobile, headless backoffice application, etc. - for the authentication flow. OAuth 2.0 is nothing if not flexible [https://oauth.net/2/grant-types/].

③ we're building a web application so the expectation is that, once you've authenticated yourself with the Spring Authorization Server, it'll redirect you back to the web application with the token in tow. But where? You specify that here. We haven't looked at the application yet, so this is a bit of foreshadowing, but the redirect URI specified here is designed to line up with Spring Security's OAuth client support, which we'll use on the web application.

④ Here we specify which scopes we'd like to be given. We've seen `openid` before, and the other two are arbitrary, and just for demonstration.

At this point, we have a valid Spring Authorization Server, and you're ready to start using it! Run the application in the usual way: `./gradlew bootRun` or `./mvnw spring-boot:run` or just run the main method from your IDE. Congratulations on your first deployment of the Spring Authorization Server. We *could* stop here, satisfied that we have got *something* to allow us to handle development chores and start building services. Indeed, if you want to, you can skip ahead and things should work fine.

Eventually, however, you're going to realize you can't leave things as they are - you'll need durable state. As-is, everything is kept in-memory. It's obviously a non-starter to have to redeploy the Spring Authorization Server every time you add a new client, user, or otherwise. People will want self-service forms by which they can register new users, clients, etc. All existing OAuth tokens would become invalid once you restart the Spring Authorization Server, too! All state related to any successful OAuth authorizations would be forgotten on every restart. The situation's not good, and in the next section, we're going to look at introducing persistence, with JDBC, to get around it.

If you want to carry on using property files, then perhaps consider the Spring Cloud Config Server. It's another piece of Spring Boot-powered middleware that, once stood up, mediates access to configuration files via an HTTP API. The configuration files live in a version control system, like Git, which the Spring Cloud Config Server monitors. When the files change, the Spring Cloud Config Server serves up the new configuration data. Even better, the Spring Cloud Config Server can, via the Spring Cloud Bus abstraction, publish notifications to your microservices (like the Spring Authorization Server) on an event bus like RabbitMQ or Apache Kafka so that you can automatically reload the new configuration. This works particularly well in tandem with the Spring Cloud's `@RefreshScope`. In such a configuration, the configuration for everything still lives in a `.properties` or `.yaml` file, as it does now, but the files are centralized and can be changed without reloading the Spring Authorization Server. Storing files in a version control system gives us niceties like versioning, auditing, rollbacks, etc., for very sensitive configuration data. And, going a step further, you can even use the Spring Authorization Server to store data encrypted at rest. For more on these possibilities, check out this video I did some years ago [https://www.youtube.com/watch?v=aC_siBP8rx8&list=PLgGXSWYM2FpPw8rV0tZoMiJYSCiLhPnOc&index=31]. And *all* of these possibilities are enabled entirely because the Spring Authorization Server is delivered as just another Spring Boot autoconfiguration! :code: ..

Persistent State in the Spring Authorization Server with PostgreSQL and JDBC

Spring makes it easy to substitute implementations with polymorphism. *Dependency injection* is one of the key reasons we use Spring. Spring Boot-based configuration makes this doubly powerful. While the Spring Authorization Server does amazing things out of the box, its real power lay in all the knobs and leavers available to you because it's just another Spring Boot autoconfiguration and starter. We have already acknowledged that keys parts of the Spring Authorization Server defer to in-memory implementations. In this section, we'll swap those, and more, out for implementations using JDBC. We'll use JDBC, but these are just interfaces. If you don't want to use JDBC, then feel free to implement the interfaces for yourself, deferring to whatever underlying storage mechanism you want. Spring Data is your friend…

Before we can get started, we'll need a PostgreSQL database. Some place in which to store our state. In the root of the project, run `docker compose up`.

Now, we're going to need to retool our project to accommodate JDBC, so add the following dependencies to the Gradle build:

```
    runtimeOnly 'org.postgresql:postgresql'
    implementation 'org.springframework.boot:spring-boot-starter-jdbc'
```

Modify `application.properties` or `application.yaml` to point to the newly configured PostgreSQL database with username, schema, and password all set to `postgres`. (Sigh. I can feel Spring Security lead Rob Winch staring at me disapprovingly because of my terribad passwords: I'm trying to demonstrate something here!) Here's the relevant configuration for `application.properties`.

```
spring.datasource.url=jdbc:postgresql://localhost/postgres
spring.datasource.username=postgres
spring.datasource.password=postgres
①
spring.sql.init.mode=always
②
spring.sql.init.schema-locations=classpath:sql/schema/*sql
③
# spring.sql.init.data-locations=classpath:sql/data/*sql
```

① This tells Spring Boot to initialize the SQL database with the schema in `src/main/resources/schema.sql` and `src/main/resources/data.sql`. Be sure to disable this property in production!

② This tells Spring Boot to not use `schema.sql` specifically, but to instead use *all* `.sql` files in `src/main/resources/sql/schema/`. This way, we can keep the various DDL statements separate.

③ If enabled, this line tells Spring Boot to not use `data.sql` specifically, but to instead use *all* `.sql` files in `src/main/resources/sql/data/`. This way, we can keep the inserts separate. Make sure to create this directory if you enable this property.

A Brief Note on Storing Credentials

Every time you see a password, or any kind of credential like a key, in the source code of these programs, remember, this is a *demonstration*. There are a million ways to externalize credentials from the source code, and you should use one of them!

It's essential to keep credentials both secure and accessible for the applications that need them, without risking exposure. Here are some methods and tools to store and make private keys accessible to a Spring Boot application:

- **Environment Variables**: One of the most common ways is to use environment variables to store sensitive data. These variables can be set on the system where your application runs. This method separates configuration from the application, but it's not the most secure method on its own, especially if multiple applications share the environment.
- **Java's KeyStore (JKS/JCEKS)**: Java provides its built-in mechanism for securely storing cryptographic keys and certificates - Java KeyStore.
- **AWS Secrets Manager or AWS Parameter Store**: If you are deploying on AWS, you can use these services to store and retrieve your application secrets.
- **HashiCorp Vault**: An open-source tool for secret management. It allows you to centrally store, access, and deploy secrets across applications and infrastructure.
- **Azure Key Vault**: If you are on Azure, you can use Azure Key Vault to store, manage, and access secrets.
- **Google Cloud Secret Manager**: For applications deployed on GCP.
- **Encrypted Configuration Files**: Use tools like Jasypt with Spring Boot. Jasypt provides Spring Boot integration and allows you to encrypt property values in your configuration files.
- **Kubernetes Secrets**: If you are deploying your application in Kubernetes, it offers its own secrets management mechanism. Though it is better than plain config maps (`ConfigMap`), Kubernetes Secrets are Base64 encoded by default and not encrypted. For enhanced security, consider integrating with external secrets managers like HashiCorp Vault.
- **Dedicated Hardware Security Modules (HSMs)**: These are physical devices that safeguard and manage digital keys, perform encryption and decryption. Cloud providers like AWS offer their own cloud HSM services.
- **Use Configuration Servers**: For example, Spring Cloud Config Server. Though not a secrets manager *per se*, when combined with encryption, it can serve configurations securely to your applications.

Always encrypt sensitive data in transit and at rest. Rotate secrets regularly. Use IAM roles, policies, and least privilege principles to restrict who can access secrets. Monitor and audit access to secrets. Avoid hardcoding secrets or placing them in a version control system (VCS), even if encrypted. Backup your secrets, but ensure backups are also secured. No matter which approach you choose, it's essential to keep the principle of least privilege in mind. Only the necessary entities should have access to your secrets, and they should only be decrypted at the last possible moment (e.g., by the application when needed).

Also, we're going to need to encode passwords so that they're not lying around at rest in plain text.

Define a bean of type `PasswordEncoder` for use in your program.

```
package com.example.authorization_server;

import com.nimbusds.jose.jwk.JWKSet;
import com.nimbusds.jose.jwk.RSAKey;
import com.nimbusds.jose.jwk.source.ImmutableJWKSet;
import com.nimbusds.jose.jwk.source.JWKSource;
import com.nimbusds.jose.proc.SecurityContext;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.factory.PasswordEncoderFactories;
import org.springframework.security.crypto.password.PasswordEncoder;

import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;

@Configuration
class SecurityConfiguration {

①
    @Bean
    PasswordEncoder passwordEncoder() {
        return PasswordEncoderFactories.createDelegatingPasswordEncoder();
    }

}
```

① this is a sort of compose `PasswordEncoder`, checking the prefix of the password string for information as to which encoder to use. We've already seen it action. The `spring encodepassword` CLI command produces a string that starts with `{bcrypt}⋯`. The default for Spring Security, today, as of this writing, is to use BCrypt. But that may change, and when the default changes, existing passwords will continue to work because the `PasswordEncoder` will know to look for the prefix and use the older BCrypt encoder when dealing with those older passwords.

We'll use the `PasswordEncoder` more later.

Persisting Users

There are other implementations of the `UserDetailsService` interface that you can use to persist users durably. Thinking from a more operational perspective, it's possible you'd want to dynamically register users dynamically, rather than having to restart the Spring Authorization Server instance after you've updated the source code. Spring Security has an extension of the `UserDetailsService` interface called `UserDetailsManager` which gives you explicit control over the lifecycle of `UserDetails`: adding, updating, deleting, etc. And, as you might imagine, there's a persistent implementation of this interface called `JdbcUserDetailsManager` that uses JDBC.

You'll need to install some SQL schema first. Spring Security ships with some usable schema on the

classpath, but unfortunately it doesn't work with PostgreSQL, and it's going to fail if there are already table definitions. So we'll modify it accordingly, as shown in `src/main/resources/sql/schema/users.sql`.

```
create table if not exists users
(
    username varchar(200) not null primary key,
    password varchar(500) not null,
    enabled  boolean      not null
);

create table if not exists authorities
(
    username  varchar(200) not null,
    authority varchar(50)  not null,
    constraint fk_authorities_users foreign key (username) references users (username
),
    constraint username_authority UNIQUE (username, authority)
);
```

We could also define the users with SQL in a file under the `sql/data/` folder, but I want you to see what it looks like to use the Java API to programmatically register a user, so here is both the `UserDetailsService` registration and a bean that uses the `UserDetailsService` to write some data to the database.

```
package com.example.authorization_server;

import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.JdbcUserDetailsManager;
import org.springframework.security.provisioning.UserDetailsManager;

import javax.sql.DataSource;
import java.util.Map;

@Configuration
class UsersConfiguration {

    @Bean
    JdbcUserDetailsManager jdbcUserDetailsManager(DataSource dataSource) {
        return new JdbcUserDetailsManager(dataSource);
    }

    @Bean
    ApplicationRunner usersRunner(PasswordEncoder passwordEncoder, UserDetailsManager
userDetailsManager) {
```

```
        return args -> {
①
            var builder = User.builder().roles("USER").passwordEncoder(
passwordEncoder::encode);
②
            var users = Map.of("josh@joshlong.com", "pw", "user@anotherone.site",
"p@ssw0rd");
            users.forEach((username, password) -> {
                if (!userDetailsManager.userExists(username)) {
                    var user = builder.username(username).password(password).build();
                    userDetailsManager.createUser(user);
                }
            });
        };
    }

}
```

① this is a convenient pattern: define the prototype for all new users once and then reuse the builder

② Poor Rob Winch's eyes! why are there passwords just strewn about our Java source code? Remember what we talked about earlier: don't do this in production code!

Persisting Clients

The `RegisteredClientRepository` interface is trivial and lends itself to implementation with a persistent store. It's easy enough to do that here, too, with implementations of the `RegisteredClientRepository`. There's an implementation called `JdbcRegisteredClientRepository` that uses JDBC to manage registered clients. It would be a fairly trivial project to implement alternatives using other persistence mechanisms like MongoDB or Hashicorp Vault.

The obvious advantage of a `RegisteredClientRepository` backed by a persistent store is that you could build a self-service registration form (or workflow) - just like Google and Apple do - for your organization developers to register clients on demand without having to restart anything or manipulate source code.

There's some schema on the classpath (`classpath:org/springframework/security/oauth2/server/authorization/client/oauth2-registered-client-schema.sql`) for the implementation that we need to take care to install first. We could tell Spring Boot to run this directly, but the trouble is that it'll fail on the second run when it executes the same DDL statements and experiences a conflict trying to create something that's already there. Create a new file `src/main/resources/sql/schema/oauth2-registered-client.sql` and use this DDL instead.

```
CREATE TABLE if not exists oauth2_registered_client
(
    id                            varchar(100)                            NOT NULL,
    client_id                     varchar(100)                            NOT NULL,
```

```
    client_id_issued_at           timestamp     DEFAULT CURRENT_TIMESTAMP NOT NULL,
    client_secret                 varchar(200)  DEFAULT NULL,
    client_secret_expires_at      timestamp     DEFAULT NULL,
    client_name                   varchar(200)                            NOT NULL,
    client_authentication_methods varchar(1000)                           NOT NULL,
    authorization_grant_types     varchar(1000)                           NOT NULL,
    redirect_uris                 varchar(1000) DEFAULT NULL,
    post_logout_redirect_uris     varchar(1000) DEFAULT NULL,
    scopes                        varchar(1000)                           NOT NULL,
    client_settings               varchar(2000)                           NOT NULL,
    token_settings                varchar(2000)                           NOT NULL,
    PRIMARY KEY (id)
);
```

And here's the definition of the `RegisteredClientRepository` and a runner that uses it to install a client, more or less identical to the client we registered earlier in `application.properties`.

```
package com.example.authorization_server;

import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.core.AuthorizationGrantType;
import org.springframework.security.oauth2.core.ClientAuthenticationMethod;
import org.springframework.security.oauth2.core.oidc.OidcScopes;
import org.springframework.security.oauth2.server.authorization.client
.JdbcRegisteredClientRepository;
import org.springframework.security.oauth2.server.authorization.client
.RegisteredClient;
import org.springframework.security.oauth2.server.authorization.client
.RegisteredClientRepository;

import java.util.Set;
import java.util.UUID;

@Configuration
class ClientsConfiguration {

①
    @Bean
    RegisteredClientRepository registeredClientRepository(JdbcTemplate template) {
        return new JdbcRegisteredClientRepository(template);
    }

②
    @Bean
    ApplicationRunner clientsRunner(PasswordEncoder pwe, RegisteredClientRepository
repository) {
```

```
        return _ -> {
            var clientId = "crm";
            if (repository.findByClientId(clientId) == null) {
                var crmClientSecret = pwe.encode("crm");
                repository.save(RegisteredClient.withId(UUID.randomUUID().toString())
                    .clientId(clientId)
                    // you should get this from an environment variable!
                    .clientSecret(crmClientSecret) ①
                    .clientAuthenticationMethod(ClientAuthenticationMethod
.CLIENT_SECRET_BASIC)
                    .authorizationGrantTypes(grantTypes -> grantTypes.addAll(
                            Set.of(AuthorizationGrantType.CLIENT_CREDENTIALS,
AuthorizationGrantType.AUTHORIZATION_CODE,
                                    AuthorizationGrantType.REFRESH_TOKEN,
AuthorizationGrantType.DEVICE_CODE)))
                    .redirectUri("http://127.0.0.1:8080/login/oauth2/code/crm")
                    .scopes(scopes -> scopes.addAll(
                            Set.of("user.read", "user.write", OidcScopes.PROFILE,
OidcScopes.EMAIL, OidcScopes.OPENID)))
                    .build());
            }
        };
    }

}
```

① register the `RegisteredClientRepository`

② this installs a registered client that's more or less equivalent to what we saw earlier in the `application.yml` properties file.

Persisting Authorizations

Remember that bit where you got redirected to the Spring Authorization Server and had to click a checkbox to confirm that the user had `user.read` scope? That fact - the consent of the scope - is stored in memory by default, but it could be stored in a database too. (Big surprise, I know!)

There are two interfaces of note here: `OAuth2AuthorizationService` and `OAuth2AuthorizationConsentService`.

`OAuth2AuthorizationService` handles representations of an authorization - the JWT token, the client, etc. It's the same story as before: there's schema on the classpath (`org/springframework/security/oauth2/server/authorization/oauth2-authorization-schema.sql`) but it doesn't work with PostgresSQL and, importantly, even if it did it would fail on the second run through because Spring Boot would try to define the table twice, in effect. So we'll modify it. Create a file `src/main/resources/sql/schema/oauth2-authorization-schema.sql`. The changes we've made replace all `blob` types with `text`.

```
create table if not exists oauth2_authorization
(
```

```
    id                            varchar(100) NOT NULL,
    registered_client_id          varchar(100) NOT NULL,
    principal_name                varchar(200) NOT NULL,
    authorization_grant_type      varchar(100) NOT NULL,
    authorized_scopes             varchar(1000) DEFAULT NULL,
    attributes                    text          DEFAULT NULL,
    state                         varchar(500)  DEFAULT NULL,
    authorization_code_value      text          DEFAULT NULL,
    authorization_code_issued_at  timestamp     DEFAULT NULL,
    authorization_code_expires_at timestamp     DEFAULT NULL,
    authorization_code_metadata   text          DEFAULT NULL,
    access_token_value            text          DEFAULT NULL,
    access_token_issued_at        timestamp     DEFAULT NULL,
    access_token_expires_at       timestamp     DEFAULT NULL,
    access_token_metadata         text          DEFAULT NULL,
    access_token_type             varchar(100)  DEFAULT NULL,
    access_token_scopes           varchar(1000) DEFAULT NULL,
    oidc_id_token_value           text          DEFAULT NULL,
    oidc_id_token_issued_at       timestamp     DEFAULT NULL,
    oidc_id_token_expires_at      timestamp     DEFAULT NULL,
    oidc_id_token_metadata        text          DEFAULT NULL,
    refresh_token_value           text          DEFAULT NULL,
    refresh_token_issued_at       timestamp     DEFAULT NULL,
    refresh_token_expires_at      timestamp     DEFAULT NULL,
    refresh_token_metadata        text          DEFAULT NULL,
    user_code_value               text          DEFAULT NULL,
    user_code_issued_at           timestamp     DEFAULT NULL,
    user_code_expires_at          timestamp     DEFAULT NULL,
    user_code_metadata            text          DEFAULT NULL,
    device_code_value             text          DEFAULT NULL,
    device_code_issued_at         timestamp     DEFAULT NULL,
    device_code_expires_at        timestamp     DEFAULT NULL,
    device_code_metadata          text          DEFAULT NULL,
    PRIMARY KEY (id)
);
```

`OAuth2AuthorizationConsentService` handles representations of an OAuth 2.0 "consent" to an Authorization request, which holds state related to the set of authorities granted to a client by the resource owner. It's the same story as before: there's schema on the classpath (`org/springframework/security/oauth2/server/authorization/oauth2-authorization-consent-schema.sql`) but it doesn't work with PostgresSQL and, importantly, even if it did it would fail on the second run through because Spring Boot would try to define the table twice, in effect. So we'll modify it. Create a file `src/main/resources/sql/schema/oauth2-authorization-consent-schema.sql`. The changes we've made replace all `blob` types with `text`.

```
create table if not exists oauth2_authorization_consent
(
    registered_client_id varchar(100)  NOT NULL,
    principal_name       varchar(200)  NOT NULL,
```

```
    authorities         varchar(1000) NOT NULL,
    PRIMARY KEY (registered_client_id, principal_name)
);
```

Persisting the HTTP Sessions Themselves

The final piece of the persistence pie is to persist the actual HTTP sessions themselves. Spring Authorization Server assumes a Servlet container is present somewhere. By default, Spring Boot uses Apache Tomcat, though that's very configurable. And Apache Tomcat, in turn, provides session management features consistent with the HTTP Servlet specification. It's even got pluggable HTTP session management, meaning you can plugin other implementations of Apache Tomcat's proprietary abstraction. But this isn't easy, or portable. Indeed, any web server is going to have some rudimentary HTTP session management, but session clustering and replication, consistency checks, etc., are not their _raison d'être: serving HTTP requests is.

Spring Session can help. It wraps the containers' default `HttpSession`. All interactions you have pass through the wrapper, which in turn delegates to any of a number of implementations backed by technology that is far faster and more reliable in the ways of making data consistently available.You can use implementations for, among other things, Hazelcast, Redis, and of course any 'ol SQL `DataSource`. We're choosing to continue to leverage our investment in PostgreSQL, so we'll use the Spring Session JDBC module.

Add the following dependency to the build:

```
    implementation 'org.springframework.session:spring-session-jdbc'
```

There is a property, `spring.session.jdbc.initialize-schema=always`, that once specified will cause Spring Session to install the JDBC schema for you in the database. It worked for me (surprise!), in PostgreSQL. But, I just really want the schema to all be in one place where I can version control it, audit it, etc, so here's the schema. Create a file `src/main/resources/sql/schema/spring-session-jdbc.sql`.

```
-- sessions
create table if not exists spring_session
(
    primary_id            character(36) primary key not null,
    session_id            character(36)             not null,
    creation_time         bigint                    not null,
    last_access_time      bigint                    not null,
    max_inactive_interval integer                   not null,
    expiry_time           bigint                    not null,
    principal_name        character varying(100)
);
create unique index if not exists spring_session_ix1 on spring_session using btree
(session_id);
create index if not exists spring_session_ix2 on spring_session using btree
(expiry_time);
```

```
create index if not exists spring_session_ix3 on spring_session using btree
(principal_name);

-- session attributes
create table if not exists spring_session_attributes
(
    session_primary_id character(36)          not null,
    attribute_name     character varying(200) not null,
    attribute_bytes    bytea                  not null,
    primary key (session_primary_id, attribute_name),
    foreign key (session_primary_id) references spring_session (primary_id)
        match simple on update no action on delete cascade
);
```

We've looked at how to persist almost all aspect of the domain of the Spring Authorization Server with JDBC. All aspects, except *keys*. Keys require a long discussion and so in the next chapter we'll look into that.

Keys

Every time the Spring Authorization Server starts up, the Spring Boot autoconfiguration kicks in generating new keys for our application. It uses the keys to sign the JWT tokens that it vends for our other applications. Other applications - clients, microservices, etc. - retain these tokens, sometimes for hours or days or even weeks! What happens when you restart the Spring Authorization Server, it autocreates new keys, and then a client with a JWT signed with the old keys tries to connect? It fails! Having the Spring Authorization Server generate random tokens on every restart can be quite a boon to getting started, but we should furnish our own, stable key if we want the JWTs to survive restarts. And load balancing, for that matter! After all, each instance of the JVM would, by default, have different keys. We're going to add two new files, a private and a public key, and use that to create a `JWKSource<SecurityContext>`.

Generate a key pair. You'll need `openssl` or `ssh-keygen` installed.

```
①
openssl genpkey -algorithm RSA -out private_key.pem
openssl pkcs8 -topk8 -inform PEM -outform PEM -in private_key.pem -out
private_key_pkcs8.pem -nocrypt
openssl rsa -pubout -in private_key_pkcs8.pem -out public_key.pem
mv private_key.pem app.key
mv public_key.pem app.pub
②
openssl req -new -x509 -key private_key_pkcs8.pem -out cert.pem -days 365
```

① make sure you execute all the following commands in a new, empty directory. It's important that some of the generated files aren't lost in the wildnerness of whatever busy folder you happen to have created them.

② this command exports a self-signed certificate (valid for 365 days) that you could (optionally) use if you wanted to store the file in the Java KeyStore as a PKCS13 file.

You'll get two artifacts, `app.pub` and `app.key`. Copy them both, the private and public key, to the `src/main/resources` folder: the private key should be called `app.key`, and the public key should be called `app.pub`.

I created three (arbitrary) properties in `application.properties` to describe these keys and assign them an ID.

```
jwt.key.id=bootiful-jwt-key-1
jwt.key.private=classpath:app.key
jwt.key.public=classpath:app.pub
```

We'll use the properties when we define the `JwkSource<ServerContext>` bean.

```
package bootiful.authorizationserver.keys;

import com.nimbusds.jose.jwk.source.JWKSource;
import com.nimbusds.jose.proc.SecurityContext;
import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import com.nimbusds.jose.jwk.RSAKey;
import com.nimbusds.jose.jwk.JWKSet;
import com.nimbusds.jose.jwk.source.ImmutableJWKSet;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;

@Configuration
class SimpleKeyConfiguration {

    @Bean
    JWKSource<SecurityContext> jwkSource(
            @Value("${jwt.key.id}") String id,
            @Value("${jwt.key.private}") RSAPrivateKey privateKey,
            @Value("${jwt.key.public}") RSAPublicKey publicKey) {
        var rsa = new RSAKey.Builder(publicKey)
                .privateKey(privateKey)
                .keyID(id)
                .build();
        var jwk = new JWKSet(rsa);
        return new ImmutableJWKSet<>(jwk);
    }

}
```

In this example, I've kept the private key (`app.key`) in the source code of this repository, but as you can imagine, this is a **very bad** idea in production. Don't forget my admonition earlier: take care to handle your credentials securely. There are some solid options, including Hashicorp Vault, your favorite hyperscaler cloud's secrets management tools, Lastpass, etc. If you need it available for this

program, it should be passed into the application in such a way that it's very difficult for others to lay hands on it. Perhaps as environment variable in your application's process space? Or as an encrypted file that can only be decrypted with a rotating key? Rotating keys… that brings up one more thing we should cover!

Rotating Keys

In the last example we plugged in a fixed key we generated by hand using `openssl`. It works, for now, but what happens when we need to rotate the keys? Rotating cryptographic keys is a well-established security practice, and it's often recommended to enhance the security posture of systems that use encryption or authentication mechanisms.

Here are a few reasons:

Regularly rotating keys limits exposure and ensures that whatever key an attacker possesses becomes obsolete after a certain time.

Key rotation also mitigates the risk of weak keys or, as sometimes happens, cryptographic erosion. Cryptographic erosion is when you have keys that are susceptible to gains in computing power that make older keys easier to compromise.

Some industry standards and regulations require periodic key rotation. For instance, the Payment Card Industry Data Security Standard (PCI DSS) has requirements around key management practices, which includes periodic key rotation.

So, we agree it's important, but how do we do it? Conceptually, it's easy: we're going to swap out the implementation of the `JWKSource` for an implementation that defers to a repository. In practice, there are a lot more moving parts involved because we have to actually persist the keys somewhere! Remember what we said about the keys in the last section? How should we encrypt the keys, rather than having them laying around on disk? The same applies here.

Also, if we're going to rotate keys, we neeed a way to generate them, and so we'll need to write some code there, too. After all, using `openssl` was easy enough for a one-time thing, but we wouldn't want to shell out for each new key generated. So, we'll need to write some code there, too. And, finally, we'll need to store these keys somewhere. I am using PostgresSQL so we'll use that.

A Brief Look at the state of Cryptography with PostgreSQL

Most databases support column-level encryption of data at rest. That is, nothing is ever written to disk in an unencrypted form. PostgreSQL does not, at least not out of the box. There are plugins, like PGCrypto [https://www.postgresql.org/docs/current/pgcrypto.html], that provide functions that you can use to encrypt text.

You can use PGCrypto pretty easily. It's a trusted module that can be loaded into the database even if you're not a superuser. Log in to your PostgreSQL instance. If you're using the `compose.yml` file, then you can run:

```
PGPASSWORD=postgres psql -U postgres -h localhoslt postgres
```

Once in, you'll need to load the plugin. It's usually bundled with your PostgreSQL distribution.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

You can then confirm that it's worked by using one of the functions provided, like this:

```
SELECT crypt('hello, world', gen_salt('bf'));
```

This example uses the Blowfish algorithm to encrypt some text. This is an awesome option, but it's unique to PostgreSQL.

Some cloud providers have PostgreSQL offerings that support transparent encryption at rest. For instance, AWS RDS for PostgreSQL supports encryption at rest using AWS Key Management Service.

I love PostgreSQL and, if I were building something for production, I'd have no trouble trusting PostgreSQL. But, in the interest of exploring Spring Security, and keeping our code as generic and easily moved from one database to another, we'll use Spring Security's equally rich encryption support to encrypt the private key (and the public one, though we don't really need to) and store it in the database. We'll need a password and a salt fort he encryption, hwoever, and those keys should ultimately not be stored at rest unencrypted. We talked about this! Store *those* keys somewhere like Hashicorp Vault or your hyperscaler's key management solution.

Generating New Keys Programatically

First thing's first: we'll need a way to generate new keys programmatically. Can't rotate 'em if we can't generate 'em! Mercifully, the Java Security APIs are pretty remarkable here.

```
package com.example.authorization_server.keys;

import org.springframework.stereotype.Component;

import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import java.time.Instant;

@Component
class Keys {

    RsaKeyPair generateKeyPair(String keyId, Instant created) {
        var keyPair = generateRsaKey();
        var publicKey = (RSAPublicKey) keyPair.getPublic();
        var privateKey = (RSAPrivateKey) keyPair.getPrivate();
        return new RsaKeyPair(keyId, created, publicKey, privateKey);
    }
```

```
    private KeyPair generateRsaKey() {
        try {
            var keyPairGenerator = KeyPairGenerator.getInstance("RSA");
            keyPairGenerator.initialize(2048);
            return keyPairGenerator.generateKeyPair();
        } //
        catch (Exception ex) {
            throw new IllegalStateException(ex);
        }
    }

}
```

Easy! The only externally visible method is `generateKeyPair`, which takes a key ID (a `kid`), and a timestamp, and returns an instance of `RsaKeyPair`, which is our repository's domain type. It's a record to hold both the generated public and private keys.

```
package com.example.authorization_server.keys;

import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import java.time.Instant;

①
record RsaKeyPair(String id, Instant created, RSAPublicKey publicKey, RSAPrivateKey
privateKey) {
}
```

① No notes. I just love Java records and want us to take a moment to appreciate them.

We're going to implement a repository to make working with, and persisting, `RsaKeyPair` instances.

```
package com.example.authorization_server.keys;

import java.util.List;

interface RsaKeyPairRepository {

①
    List<RsaKeyPair> findKeyPairs();

②
    void save(RsaKeyPair rsaKeyPair);

}
```

① Find the key pairs and preserve ordering. We want the latest key to be first.

② This method looks so simple. Don't trust it. It's a lie!

Saving the `RsaKeyPair`, in this example, means writing it to our database, which means we'll need a way to serialize the `java.security.*` key objects. It's not as straightforward as you'd think! But I got it working and put that logic in two implementations of Spring Framework's handy `Converter<T>` interface: `RsaPrivateKeyConverter` for private keys, and `RsaPublicKeyConverter` for public keys.

Here's the `RsaPrivateKeyConverter`:

```
package com.example.authorization_server.keys;

import org.springframework.core.serializer.Deserializer;
import org.springframework.core.serializer.Serializer;
import org.springframework.security.crypto.encrypt.TextEncryptor;
import org.springframework.util.FileCopyUtils;

import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.security.KeyFactory;
import java.security.interfaces.RSAPrivateKey;
import java.security.spec.PKCS8EncodedKeySpec;
import java.util.Base64;

class RsaPrivateKeyConverter implements Serializer<RSAPrivateKey>, Deserializer
<RSAPrivateKey> {

    private final TextEncryptor textEncryptor;

    RsaPrivateKeyConverter(TextEncryptor textEncryptor) {
        this.textEncryptor = textEncryptor;
    }

①
    @Override
    public void serialize(RSAPrivateKey object, OutputStream outputStream) throws
IOException {
        var pkcs8EncodedKeySpec = new PKCS8EncodedKeySpec(object.getEncoded());
        var string = "-----BEGIN PRIVATE KEY-----\n"
                + Base64.getMimeEncoder().encodeToString(pkcs8EncodedKeySpec
.getEncoded())
                + "\n-----END PRIVATE KEY-----";
        outputStream.write(this.textEncryptor.encrypt(string).getBytes());
    }

②
    @Override
    public RSAPrivateKey deserialize(InputStream inputStream) {
        try {
            var pem = this.textEncryptor.decrypt(FileCopyUtils.copyToString(new
InputStreamReader(inputStream)));
```

```
            var privateKeyPEM = pem.replace("-----BEGIN PRIVATE KEY-----", "").
replace("-----END PRIVATE KEY-----", "");
            var encoded = Base64.getMimeDecoder().decode(privateKeyPEM);
            var keyFactory = KeyFactory.getInstance("RSA");
            var keySpec = new PKCS8EncodedKeySpec(encoded);
            return (RSAPrivateKey) keyFactory.generatePrivate(keySpec);
        } //
        catch (Throwable throwable) {
            throw new IllegalArgumentException("there's been an exception", throwable
);
        }
    }

}
```

① serialization involves adding a header and footer to the content of the key and then serializing, but not before Base64 encoding the content of the key, and then passing it through a Spring Security `TextEncryptor`

② deserialization is basically the same, in reverse. Decrypt the text by passing it through a Spring Security `TextEncryptor`, then strip out the header and footer, and then use the Base64 decoder to decode the content, and then finally create a key using the Java Security `KeyFactory`.

The code for `RsaPublicKeyConverter`. We won't rehash it since its structure is basically identical to the `RsaPrivateKeyConverter`.

```
package com.example.authorization_server.keys;

import org.springframework.core.serializer.Deserializer;
import org.springframework.core.serializer.Serializer;
import org.springframework.security.crypto.encrypt.TextEncryptor;
import org.springframework.util.FileCopyUtils;

import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.security.KeyFactory;
import java.security.interfaces.RSAPublicKey;
import java.security.spec.X509EncodedKeySpec;
import java.util.Base64;

class RsaPublicKeyConverter implements Serializer<RSAPublicKey>, Deserializer
<RSAPublicKey> {

    private final TextEncryptor textEncryptor;

    RsaPublicKeyConverter(TextEncryptor textEncryptor) {
        this.textEncryptor = textEncryptor;
    }
```

```
    @Override
    public RSAPublicKey deserialize(InputStream inputStream) throws IOException {
        try {
            var pem = textEncryptor.decrypt(FileCopyUtils.copyToString(new
InputStreamReader(inputStream)));
            var publicKeyPEM = pem.replace("-----BEGIN PUBLIC KEY-----", "").replace("
-----END PUBLIC KEY-----", "");
            var encoded = Base64.getMimeDecoder().decode(publicKeyPEM);
            var keyFactory = KeyFactory.getInstance("RSA");
            var keySpec = new X509EncodedKeySpec(encoded);
            return (RSAPublicKey) keyFactory.generatePublic(keySpec);
        } //
        catch (Throwable throwable) {
            throw new IllegalArgumentException("there's been an exception", throwable
);
        }

    }

    @Override
    public void serialize(RSAPublicKey object, OutputStream outputStream) throws
IOException {
        var x509EncodedKeySpec = new X509EncodedKeySpec(object.getEncoded());
        var pem = "-----BEGIN PUBLIC KEY-----\n"
                + Base64.getMimeEncoder().encodeToString(x509EncodedKeySpec.
getEncoded())
                + "\n-----END PUBLIC KEY-----";
        outputStream.write(this.textEncryptor.encrypt(pem).getBytes());
    }

}
```

We'll look at the configuration of the `TextEncryptor` used in both implementations in a bit.

We'll need to register these converters. I suppose I could've aded a `@Component` annotation to each of them, but instead I chose to register them with Java configuration, like this:

```
package com.example.authorization_server.keys;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.encrypt.Encryptors;
import org.springframework.security.crypto.encrypt.TextEncryptor;

@Configuration
class Converters {

    @Bean
    RsaPublicKeyConverter rsaPublicKeyConverter(TextEncryptor textEncryptor) {
```

```
        return new RsaPublicKeyConverter(textEncryptor);
    }

    @Bean
    RsaPrivateKeyConverter rsaPrivateKeyConverter(TextEncryptor textEncryptor) {
        return new RsaPrivateKeyConverter(textEncryptor);
    }

}
```

Let's look at the repository implementation which will use JDBC.

```
package com.example.authorization_server.keys;

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Component;
import org.springframework.util.Assert;

import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.util.Date;
import java.util.List;

@Component
class JdbcRsaKeyPairRepository implements RsaKeyPairRepository {

    private final JdbcTemplate jdbc;

    private final RsaPublicKeyConverter rsaPublicKeyConverter;

    private final RsaPrivateKeyConverter rsaPrivateKeyConverter;

    private final RowMapper<RsaKeyPair> keyPairRowMapper;

    JdbcRsaKeyPairRepository(RowMapper<RsaKeyPair> keyPairRowMapper,
RsaPublicKeyConverter publicConverter,
            RsaPrivateKeyConverter privateConverter, JdbcTemplate jdbc) {
        this.jdbc = jdbc;
        this.keyPairRowMapper = keyPairRowMapper;
        this.rsaPublicKeyConverter = publicConverter;
        this.rsaPrivateKeyConverter = privateConverter;
    }

①
    @Override
    public List<RsaKeyPair> findKeyPairs() {
        return this.jdbc.query("select * from rsa_key_pairs order by created desc",
this.keyPairRowMapper);
    }
```

```
②
    @Override
    public void save(RsaKeyPair keyPair) {
        var sql = """
                insert into rsa_key_pairs (id, private_key, public_key, created)
                values (?, ?, ?, ?)
                on conflict on constraint rsa_key_pairs_id_created_key
                do nothing
                """;
        try (var privateBaos = new ByteArrayOutputStream(); var publicBaos = new
ByteArrayOutputStream()) {
            this.rsaPrivateKeyConverter.serialize(keyPair.privateKey(), privateBaos);
            this.rsaPublicKeyConverter.serialize(keyPair.publicKey(), publicBaos);
            var updated = this.jdbc.update(sql, keyPair.id(), privateBaos.toString(),
publicBaos.toString(),
                    new Date(keyPair.created().toEpochMilli()));
            Assert.state(updated == 0 || updated == 1, "no more than one record should
have been updated");
        } //
        catch (IOException e) {
            throw new IllegalArgumentException("there's been an exception", e);
        }
    }

}
```

① in order to find the records we'll issue a query and pass in the `RowMapper<RsaKeyPair>`, which is an instance of `RsaKeyPairRowMapper`, which we'll explore shortly.

② writing the record is pretty easy, too. The converters do most of the work. then it's a simple matter of lining up the converted values as arguments for the SQL update.

We saw that the repository implementation uses a Spring JDBC `RowMapper<T>` instance, which looks like this:

```
package com.example.authorization_server.keys;

import org.springframework.core.serializer.Deserializer;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Date;

@Component
class RsaKeyPairRowMapper implements RowMapper<RsaKeyPair> {

    private final RsaPrivateKeyConverter rsaPrivateKeyConverter;
```

```
    private final RsaPublicKeyConverter rsaPublicKeyConverter;

    RsaKeyPairRowMapper(RsaPrivateKeyConverter rsaPrivateKeyConverter,
RsaPublicKeyConverter rsaPublicKeyConverter) {
        this.rsaPrivateKeyConverter = rsaPrivateKeyConverter;
        this.rsaPublicKeyConverter = rsaPublicKeyConverter;
    }

    @Override
    public RsaKeyPair mapRow(ResultSet rs, int rowNum) throws SQLException {
        try {

①
            var privateKey = loadKey(rs, "private_key", this.rsaPrivateKeyConverter);
            var publicKey = loadKey(rs, "public_key", this.rsaPublicKeyConverter);

②
            var created = new Date(rs.getDate("created").getTime()).toInstant();
            var id = rs.getString("id");

③
            return new RsaKeyPair(id, created, publicKey, privateKey);
        }
        catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

    private static <T> T loadKey(ResultSet rs, String fn, Deserializer<T> f) throws
SQLException, IOException {
        var privateKeyBytes = rs.getString(fn).getBytes();
        return f.deserializeFromByteArray(privateKeyBytes);
    }

}
```

① again, one of the nice things about this implementation is that most of the heavy lifting is in the converters.

② this stuff is thankfully much easier to deal with

③ I've extracted out the logic of extracting a field from theh `ResultSet`, passing it to a converter, and then returning it to a separate method.

now we've done everything we need to do to support reading and writing key pairs. Let's put it all to good use in an implementation of `JWKSource`, which - when you look at it - is almost anticlimactic. There are two concerns being adressed in this implementation, telling Nimbus, the library whose support for JWT underpins the Spring Authorization Server, how to load a new key given a query (called a `JWKSelector`) and telling Spring Authorization Server that we want to have a chance to post-process, to *customize*, the OAuth 2.0 token attributes. We'll plug that OAuth2TokenCustomizer

functionality in later.

```
package com.example.authorization_server.keys;

import com.nimbusds.jose.KeySourceException;
import com.nimbusds.jose.jwk.JWK;
import com.nimbusds.jose.jwk.JWKSelector;
import com.nimbusds.jose.jwk.RSAKey;
import com.nimbusds.jose.jwk.source.JWKSource;
import com.nimbusds.jose.proc.SecurityContext;
import
org.springframework.security.oauth2.server.authorization.token.JwtEncodingContext;
import
org.springframework.security.oauth2.server.authorization.token.OAuth2TokenCustomizer;
import org.springframework.stereotype.Component;

import java.util.ArrayList;
import java.util.List;

@Component
class RsaKeyPairRepositoryJWKSource implements JWKSource<SecurityContext>,
OAuth2TokenCustomizer<JwtEncodingContext> {

    private final RsaKeyPairRepository keyPairRepository;

    RsaKeyPairRepositoryJWKSource(RsaKeyPairRepository keyPairRepository) {
        this.keyPairRepository = keyPairRepository;
    }

①
    @Override
    public List<JWK> get(JWKSelector jwkSelector, SecurityContext context) throws
KeySourceException {
        var keyPairs = this.keyPairRepository.findKeyPairs();
        var result = new ArrayList<JWK>(keyPairs.size());
        for (var keyPair : keyPairs) {
            var rsaKey = new RSAKey.Builder(keyPair.publicKey()).privateKey(keyPair
.privateKey())
                .keyID(keyPair.id())
                .build();
            if (jwkSelector.getMatcher().matches(rsaKey)) {
                result.add(rsaKey);
            }
        }
        return result;
    }

②
    @Override
    public void customize(JwtEncodingContext context) {
```

```
        var keyPairs = this.keyPairRepository.findKeyPairs();
        var kid = keyPairs.get(0).id();
        context.getJwsHeader().keyId(kid);
    }

}
```

① when asked, we pass thrugh and return the list of `RsaKeyPair` instances from the database, turning them into `RSAKey.Builder` instances.

② when asked, we customize the JWTs that are generated by specifying the key ID, so that it lines up with the keys in the repository.

Let's see how all of this gets plugged into Spring Authorization Server through configuration.

```
package com.example.authorization_server.keys;

import com.nimbusds.jose.jwk.source.JWKSource;
import com.nimbusds.jose.proc.SecurityContext;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.encrypt.Encryptors;
import org.springframework.security.crypto.encrypt.TextEncryptor;
import org.springframework.security.oauth2.core.OAuth2Token;
import org.springframework.security.oauth2.jwt.JwtEncoder;
import org.springframework.security.oauth2.jwt.NimbusJwtEncoder;
import org.springframework.security.oauth2.server.authorization.token.*;

@Configuration
class KeyConfiguration {

①
    @Bean
    TextEncryptor textEncryptor(@Value("${jwk.persistence.password}") String pw,
            @Value("${jwk.persistence.salt}") String salt) {
        return Encryptors.text(pw, salt);
    }

②
    @Bean
    OAuth2TokenGenerator<OAuth2Token> delegatingOAuth2TokenGenerator(JwtEncoder
encoder,
            OAuth2TokenCustomizer<JwtEncodingContext> customizer) {
        var generator = new JwtGenerator(encoder);
        generator.setJwtCustomizer(customizer);
        return new DelegatingOAuth2TokenGenerator(generator, new
OAuth2AccessTokenGenerator(),
                new OAuth2RefreshTokenGenerator());
    }
```

```
③
    @Bean
    NimbusJwtEncoder jwtEncoder(JWKSource<SecurityContext> jwkSource) {
        return new NimbusJwtEncoder(jwkSource);
    }

}
```

① this is the `TextEncryptor` that's doing so much of the work for us in the converters we looked at earier. It's a `TextEncryptor`, as opposed to a `BytesEncryptor`. Ultimately, it's using an algorithm called AES, which is the gold standard in ciphers today, succeeding a long line of other algorithms - like DES and XDES - that go all the way back to the 1970's. It's both space efficient and cryptographically secure.

② We only want to plugin the `OAuth2TokenCustomizer` for the `JwtGeneator`, but need to redeclare the bean that uses it, the `OAuth2TokenGenerator`. most of this is boilerplate that we're copying from the defaults.

③ and finally we want to plugin our new `JWKSource` implementation
NOTE

> In this last example we injected some propertues, `jwk.persistence.password` and `jwk.persistence.salt`, which are credentials that need to be stored in a secure fashion!

The machinery is in place, but something needs to start it! We want the application to startup and automatically register a new `RsaKeyPair` if none exists in the database (otherwise the Spring Authorization Server wouldn't work!), and we want to be able to rotate the keys automatically. So, I've created a new Spring `ApplicationEvent`, called `RsaKeyPairGenerationRequestEvent`. We'll rig up a listener to rotate the keys whenever an instance of that event is published.

Any code anywhere in the Spring Authorization Server could publish that event and trigger a key rotation. You could have a `@Scheduled` method that runs every 24 hours and rotates the keys. You could create a (secured) HTTP endpoint that publishes the event. You could create a (secured) Actuator endpoint that publishes the event. You could write some code to listen to new messages coming in from Kafka and then publish the event in response. The skies the limit! The first we're going to publish the event, howeve? On startup, but only if we discover there are no keys in the repository. Let's see that event wiring.

```
package com.example.authorization_server.keys;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.context.event.ApplicationReadyEvent;
import org.springframework.context.ApplicationEventPublisher;
import org.springframework.context.ApplicationListener;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.time.Instant;

@Configuration
class LifecycleConfiguration {
```

```
①
    @Bean
    ApplicationListener<RsaKeyPairGenerationRequestEvent>
keyPairGenerationRequestListener(Keys keys,
            RsaKeyPairRepository repository, @Value("${jwk.key.id}") String keyId) {
        return event -> repository.save(keys.generateKeyPair(keyId, event.getSource()
));
    }

②
    @Bean
    ApplicationListener<ApplicationReadyEvent> applicationReadyListener
(ApplicationEventPublisher publisher,
            RsaKeyPairRepository repository) {
        return event -> {
            if (repository.findKeyPairs().isEmpty())
                publisher.publishEvent(new RsaKeyPairGenerationRequestEvent(Instant
.now()));
        };
    }

}
```

① this `ApplicationListener` listens for the aforementioned event and writes a new `RsaKeyPair` to the repository, using the injeted `jwk.key.id`, whcih should be specified externally, and should remain constant. that is, after all, the key ID.

② this `ApplicationListener` runs when the service starts and publishes a `RsaKeyPairGenerationRequestEvent`, but only if there are no `RsaKeyPairs` in teh repository already.

At this point you can delete the key files we generated with `openssl` earlier. You can also delete the relevant configration in your `application.properties` or `application.yml`; the application can now create and rotate its own keys.

To Production… and Beyond!

And that's it! Restart the Spring Authorization Server. Now you can run more than one instance of the Spring Authorization Server behind a load balancer, and no matter to which instance a client and its cookies present themselves, the container will resolve the session, authorizations, information about clients from the PostgreSQL database. It'll also sign keys in a consistent fashion. Is now a good time to remind you not to stash sensitive stuff in the HTTP session? You never know when it's going to be serialized to some datastore… = Federated OAuth with the Spring Authorization Server

Secure Your Services

The Gateway Client

In our super secure OAuth onion, this next microservice, the `gateway`, is the outermost layer. It is the first port-of-call for all requests destined for the microservices in our system. When visitors to our site punch in the domain name for our system into a browser, it'll resolve to a loadbalancer serving this gateway service, and it is here where we'll originate an OAuth token. This service is also a gateway, powered by Spring Cloud Gateway. The gateway acts as a proxy, allowing us to forward requests to different hosts and ports, concealing from the user that the responses they're seeing are from different services.

The `gateway` service's job is entirely to handle the OAuth dance - if it detects that the request is not authenticated - and to otherwise proxy requests to two other HTTP endpoints: the `api` we just built, and have running on port `8081`, and the static HTML process running on port `8020`.

Here's the routing table

HTTP origin	HTTP destination
http://gateway:8082/api/customers	http://api:8080/customers
http://gateway:8082/api/me	http://api:8080/me
http://gateway:8082/api/email	http://api:8080/email
http://gateway:8082/index.html	http://static:8020/index.html

The `gateway` also sends along the JWT token to the downstream HTTP endpoints, acting as a token relay. The `static` site won't care (it's just serving up static `.html` and `.css` files, after all), but the `api` is a resource server, and it will care. The `api` will refuse to send back data unless that token is specified.

Let's look at the Java code first.

```
package com.example.gateway_reactive;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.web.server.ServerHttpSecurity;
import org.springframework.security.web.server.SecurityWebFilterChain;

@Configuration
class SecurityConfiguration {

①
    @Bean
    SecurityWebFilterChain securityWebFilterChain(ServerHttpSecurity http) {
        http
```

```
                .authorizeExchange((authorize) -> authorize.anyExchange()
.authenticated())②
                .csrf(ServerHttpSecurity.CsrfSpec::disable)③
                .oauth2Login(Customizer.withDefaults())④
                .oauth2Client(Customizer.withDefaults());
        return http.build();
    }


}
```

① you could probably get away with not defining this bean at all *if* you were willing to deal with default behavior of CSRF tokens, which I am not. In the interest of simplicity, I'm disabling them, but in so doing I am also disabling all the other things Spring Security assumed I wanted. So we will also go through and re-enable those things.

② We want all HTTP requests to be authenticated…

③ …and to disable the CSRF support…

④ …and to re-enable OAuth 2 OIDC login support and the OAuth 2 client support. The OIDC login functionality is what triggers the OAuth dance we've talked about. The OAuth 2 client support is what tells Spring Security, running in this process, as which OAuth 2 client requests for OIDC login should be done. We'll need to specify the particulars in the property configuration later.

The security configuration is pretty straightforward. We're an OAuth client. We want to prompt users to login with a particular client. Once a user is authenticated, well, there's not much for them to see! We need Spring Cloud Gateway to connect our other HTTP services to the user visiting this `gateway` service.

We'll configure two Spring Cloud Gateway *routes*. Each request has a predicate, optional filter(s), and a destination URI. The predicate defines how requests to the Spring Cloud Gateway service are matched, e.g.: does this request have a particular header or cookie, or a particlar path, or a particular virtual host? You may specify one or more filters that act on the incoming requests, changing it. Finally, the request, after it has passed through any and all filters, is sent to a final destination, which we specify with a URI.

```
package com.example.gateway_reactive;

import org.springframework.cloud.gateway.route.RouteLocator;
import org.springframework.cloud.gateway.route.builder.RouteLocatorBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
class GatewayConfiguration {

    @Bean
    RouteLocator gateway(RouteLocatorBuilder rlb) {
        var apiPrefix = "/api/";
        return rlb
```

```
                .routes()
①
                .route(rs -> rs
                        .path(apiPrefix + "**")
                        .filters(f -> f
                                .tokenRelay()
                                .rewritePath(apiPrefix + "(?<segment>.*)", "/$
\\{segment}")
                        )
                        .uri("http://localhost:8081"))
②
                .route(rs -> rs
                        .path("/**")
                        .uri("http://localhost:8020")
                )
                .build();
    }

}
```

① the first route matches all requests to `/api/**`, notes and forwards any OAuth JWTs to the backend service, and changes the path of the request from `gateway:8082/api/foo` to `api:8081/foo`, dropping the `/api/` bit.

② the second route takes every other request and sends it unchecked on to the HTTP endpoint service up the static HTMl 5 and JavaScript assets.

That's just about all the Java code for this service, but its role and importance in the architecture can not be overstated. Let's look at the property file that ties it all together.

```
spring.application.name=gateway-reactive
①
server.port=8082
②
spring.security.oauth2.client.provider.spring.issuer-uri=http://localhost:8080
③
spring.security.oauth2.client.registration.spring.provider=spring
spring.security.oauth2.client.registration.spring.client-id=crm
spring.security.oauth2.client.registration.spring.client-secret=crm
spring.security.oauth2.client.registration.spring.authorization-grant-
type=authorization_code
spring.security.oauth2.client.registration.spring.client-authentication-
method=client_secret_basic
spring.security.oauth2.client.registration.spring.redirect-
uri={baseUrl}/login/oauth2/code/{registrationId}
spring.security.oauth2.client.registration.spring.scope=user.read,openid
```

① this process will run on port `8082`.

② it will use the issuer URI to validate JWTs if it detects them

③ otherwise it will use this configuration to, acting as a client, inintiate an OIDC login to allow you to come up with a valid token.

The last several lines are where we tell the OAuth client what it should ask for from our OAuth IDP (the Spring Authorization Server). You'd write similar configuration for any OAuth IDP, not just the Spring Authorization Server. In this example, the client is configured to identify itself as the `crm` client, with `crm` as the client secret. It's being configured to ask for the `authorization_code` authorization grant. It's being configured to instruct the OAuth IDP to rediect *back* to the gateway, with a special pseudo URL syntax: `{baseUrl}/login/oauth2/code/{registrationId}`. Spring Security will set that endpoint up for us on the gateway, so there's nothing we need to do for that to work. Finally, the OAuth client asks for certain permissions: `user.read` and `openid`.

A User Interface for a Browser User

The OAuth client is where all outside visitors will begin their journey. But obviously, they're not going to be issuing HTTP requests with `curl` , they're going to be expecting some sort of user interface. We looked at how Spring Cloud Gateway requests will route proxied resources: `/api/**` to our backend `api` and everything else to the static HTML5 and JavaScript code. In this case, it's just running on our local machines, but one images this stuff would be deployed to a CDN and made available locally in a real production application.

In order to keep things as simple as possible, I've written a snigle page `.html` application with a smattering - a pinch, a dash, even! - of JavaScript. The concepts will be the same whether you eventually use Vue.js, React, Angular, jQuery, or whatever other thing you decide to use.

Here's the `.html` file. It has a panel to greet the signed in user and another to show a grid of `customer` records.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>Spring CRM</title>
    <link rel="text/css" href="app.css"/>
</head>
<body>
<script src="app.js"></script>
<div>welcome, <span style="font-weight: bold" id="me"></span></div>
<div id="customers"></div>
</body>
</html>
```

Both the user name and the `customer` records we'll load by talking to the API from JavaScript.

```
①
const root = '/api'
```

```
async function customers() {
    const response = await fetch(root + '/customers')
    return await response.json()
}

async function me() {
    const response = await fetch(root + '/me')
    return await response.json()
}


②
async function email(customerId) {
    const response = await fetch(root + '/email?customerId=' + customerId, {method:
'POST'})
    const data = await response.json()
    const cssQuerySelector = '#' + divIdFor({id: customerId}) + ' .id'
    document.querySelector(cssQuerySelector).classList.add('fade')
    return data
}

③
function divIdFor(customer) {
    return 'customerDiv' + customer.id
}

④
window.addEventListener('load', async (event) => {
    document.getElementById('me').innerHTML = (await me()).name;
    const customersDiv = document.getElementById('customers')
    const customersResults = await customers();
    customersResults.forEach(customer => {
        const div = document.createElement('div')
        div.innerHTML = `
              <button type="button">email report</button>
              <span class="id"> ${customer.id} </span>
              <span> ${customer.name}</span>
            `
        div.id = divIdFor(customer)
        customersDiv.appendChild(div)
        document
            .querySelector('#' + divIdFor(customer))
            .addEventListener('click', () => email(customer.id))
    });
})
```

① the first two functions of the file do the work of interacting with our API using the non-blocking, but oh-so-verbose, `async/await` syntax. (If only JavaScript had Java 21's Project Loom and virtual threads!). NB: we're making these requests relative to the `/api` path. We've written the JavaScript code in such a way that it knows that it's being accessed via an HTTP proxy, our Spring Cloud

Gateway gateway.

② this function is a little more interesting in that we're also interacting with the backend API but we're sending `POST` calls, not `GET` calls. And of course we've got a little animation goin' on.

③ there are at least two separate places that I need the `id` that'll have been assigned to a `div` for records for a particular `customer`. So I've extracted it into a sepearate function here.

④ this is the heart of the code: when the page loads, we read the data for the customers and the user's name and then draw the page with some DOM manipulation. This code also wires up an event so that when you click on one of the rendered buttons, it triggers a `POST` which initiates a message sent to RabbitMQ.

I love this code, not because of the JavaScript (bleargh!), but because it's so plain to understand. Remember, this code will run in a browser that has cookies associated with an HTTP session, and that HTTP session in turn is connected with a valid OAuth token. So each time the JavaScript code calls `/api/customers`, it's implicitly sending the associated cookie to the gateway code, which in turn then allows the gateway to find the OAuth session associated with this user and to then forward the token to the backend `api`.

At no point does the JavaScript code ever see your username and password. And, in this case, it doesn't even see your JWT! The code is written in such a way that it doesn't even know OAuth is involved. Now you can develop the application without OAuth, get it working, then add the OAuth resource server and OAuth client support to the backend code and call it done.

Remember, the user visiting the HTML page won't even see any of this markup unless they've authenticated. They'll be redirected away before they ever get a chance. This page is both completely and blissfully unaware of any security, and it doesn't need to be aware of security.

=

Secure Everything Else

Appendix A: Appendix

Your First Cup of Java

Are you using the latest version of Java yet? No? That's a real pity, because it's basically a different language from the one you might've known. This book was written with Java 25 in mind, but you should be using whatever the latest-and-greatest version of Java available to you is.

That said, i can appreciate that some of you may not have seen all that is new and nifty in the latest and greatest installments of Java, so we'll review some of my favorite features here. If you're all caught up, then please move on to the next chapters - there's entirely too much to look at, anyway.

I don't know what the reason is, but I hope you're able to move up and over soon because Spring Framework 6 and Spring Boot 3 assume a Java 17 baseline. That means that not only do those generations of Spring support Java 17 features (as did the Spring Framework 5 and Spring Boot generations), they also use some Java 17 features in their source code. This book uses Spring Framework 6 and Spring Boot 3.

Whatever the cause of your hesitation, this chapter contains *spoilers*! If you don't mind spoilers, let's dive right in!

First things first: you need an OpenJDK distribution that works for you. The list of valid and viable distributions scrolls down to the floor and out the door, so I'll refer you to Fooojay.io's handy version almanac for Java 17 [https://foojay.io/almanac/java-17/]. One of my favorite tools for managing my Java installations on UNIX-y type operating systems is `sdkman` [https://sdkman.io/]. It works on macOS, Windows Subsystem for Linux (WSL), Linux, and I'm sure other places besides. I use the GraalVM [https://www.graalvm.org/] distribution. GraalVM is an OpenJDK distribution with some extra features, including ahead-of-time native image compilation. To get that distribution, you might say:

```
sdk install java 25-graalce ①
sdk default java 25-graalce ②
```

① the first command installs the latest version of the GraalVM distribution of Java 25

② the second makes it the default installed distribution on my box so that all my interactions with Java are going to that distribution

With that done, let's look at some neat features in Java's latest and greatest versions up until Java 25.

Operations Improvements

The newer versions of Java now support something called CDS Archives. CDS Archives essentially capture some of the invariant (but freshly computed) state from a given application and cache it for easy reuse on subsequent runs. I consistently shave 0.1 or 0.2 seconds from startup time when using CDS Archives.

The newer versions of Java are also container-aware. So, let's suppose you are running Docker

images on your host machine, which has 32GB of RAM. You might configure the JRE with 2GB of RAM and errantly configure the Docker container with only 1GB of RAM. Java would see the 32GB and think it could allocate 2GB and then fail to startup. Java is aware of the container's limited RAM in newer versions and won't exceed it.

Java Flight Recorder (JEP 328) monitors and profiles running Java applications. Even better, it does so with a low runtime footprint. Java Mission Control allows ingesting and visualizing Java Flight Recorder data. Java Mission Control takes Java's already stellar support for observability to the next level.

Performance

Java 25 is *fast* and reliable.

Java's garbage collector is the stuff of legend. It's fast, lightweight, and minimally invasive. It's also one of those things where, when it's improved, your application's runtime improves with it, for free. No recompilation is required.

G1 has been the default garbage collector since Java 9, replacing the Parallel garbage collector in Java 8. It reduces pause times with the default Parallel GC from Java 8, though it may have lower throughput overall. Next, Java 11 introduced the ZGC garbage collector to reduce pauses further. Finally, Java 14 introduced the Shenandoah GC, which keeps pause times low and does so in a manner independent of the heap's size.

And it's fast. I can't give you a specific number or anything because it is so highly workload-sensitive, but this post from the folks at OptaPlanner [https://www.optaplanner.org/blog/2025/09/15/HowMuchFasterIsJava17.html] is persuasive. They saw an average of 8.66% improvement for their CPU-intensive workloads when using the G1 GC, measured after a discarded 30 second warmup period. These numbers reflect the jump from Java 11 (not Java 8) to Java 17. I can only imagine the numbers from Java 8 to Java 17 are even better. And the numbers for Java 25, even better still! And that number is just an average: some workloads improved by as much as 23%!

Autocloseable

Java manages most memory for you, but it can't be responsible for the state outside the humble confines of the JVM. So, for example, you wouldn't like it if Java *garbage collected* your connection to the database, and you wouldn't like it if Java garbage-collected your open socket connections without warning. Therefore, the interfaces you use to interact with these external resources typically have a `close()` method that you, the client, need to call when working the external resource finishes.

You mustn't neglect to call that method! Don't be greedy! You're not greedy, are ya? You want to leave the JVM as clean as you found it. So, you write the boilerplate. We all know the boilerplate: open the resource; work with it; surround that work with a `try/catch/finally` block; catch any `Throwable` instances if (*when*?) something goes wrong; `close()` the resource if something goes wrong. Add the `close()` call in a `finally` block for extra robustness. All the while, handle the exceptions that might arise when you try to do anything, including calling `close()` the resource. You'll end up with one or more `try/catch` embedded within the outer `try/catch` blocks.

Let's look at some examples. I'll read a file on the filesystem to demonstrate this new feature. But something needs to ensure that there's a file to read in the first place, so I've extracted all that out into a separate class, `Utils`:

```
package com.example.java.closeable;

import java.io.File;
import java.nio.file.Files;
import java.time.Instant;

public class Utils {

①
    static String CONTENTS = String.format("""
            <html>
            <body><h1> Hello, world, @ %s !</h1> </body>
            </html>
            """, Instant.now().toString()).trim();

②
    static File setup() {
        try {
            var path = Files.createTempFile("bootiful", ".txt");
            var file = path.toFile();
            file.deleteOnExit();
            Files.writeString(path, CONTENTS);
            return file;
        } //
        catch (Throwable t) {
            error(t);
            throw new RuntimeException("could not produce a new file");
        }

    }

③
    static void error(Throwable throwable) {②
        System.err.println("there's been an exception processing the read! " +
throwable.getMessage());
    }

}
```

① the contents of the file, using a handy multiline Java `String`

② this method returns the `java.io.File` for the newly created temporary file

③ a convenience method to log errors. Do not rewrite this code! Otherwise, if you're doing things the old-fashioned way, you'll find yourself rewriting this a *lot*.

Now, let's look at an example of what *not* to do.

```
package com.example.java.closeable;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.io.*;

import static com.example.java.closeable.Utils.error;

class TraditionalResourceHandlingTest {

    private final File file = Utils.setup();①

    private static void close(Reader reader) { ③
        if (reader != null) {
            try {
                reader.close();
            } //
            catch (IOException e) {
                error(e);
            }
        }
    }

    @Test
    void read() {
        var bufferedReader = (BufferedReader) null; ②
        try {
            bufferedReader = new BufferedReader(new FileReader(this.file));
            var stringBuilder = new StringBuilder();
            var line = (String) null;
            while ((line = bufferedReader.readLine()) != null) {
                stringBuilder.append(line);
                stringBuilder.append(System.lineSeparator());
            }
            var contents = stringBuilder.toString().trim();
            Assertions.assertEquals(contents, Utils.CONTENTS);
        } //
        catch (IOException e) { ③
            error(e);
        } //
        finally { ④
            close(bufferedReader);
        }
    }

}
```

① from which file are we reading?

② We need a reference to the `BufferedReader` outside of the scope of the `try/catch` block, but we don't want to initialize that reference until we're inside the `try/catch` block because it might incur an exception.

③ handle any errors. Bear in mind that this solution doesn't even attempt to field the errors and somehow recover. If there is *any* error anywhere, then we abort.

④ also, make sure to close that `Reader`! Err, close that reader *if* it's not `null`! Sorry, close that reader *if* it's not `null` and also don't forget to handle any errors in the doing, either! Be kind, rewind!

Yuck. We can do better with Java 7's `try-with-resources` construct. Let's rework the example accordingly.

```
package com.example.java.closeable;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;

class TryWithResourcesTest {

    private final File file = Utils.setup();

    @Test
    void tryWithResources() {
        try (var fileReader = new FileReader(this.file); //
                var bufferedReader = new BufferedReader(fileReader)) {
            var stringBuilder = new StringBuilder();
            var line = (String) null;
            while ((line = bufferedReader.readLine()) != null) {
                stringBuilder.append(line);
                stringBuilder.append(System.lineSeparator());
            }
            var contents = stringBuilder.toString().trim();
            Assertions.assertEquals(contents, Utils.CONTENTS);
        } //
        catch (IOException e) {
            Utils.error(e);
        }
    }

}
```

Type Inference

Java 10 introduced type inference for variables given enough context. As a result, Java is a strongly

typed language with less typing - pressing of keys - required.

```
package com.example.java.typeinference;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.Map;

class TypeInferenceTest {

    @Test
    void infer() throws Exception {
        var map1 = Map.of("key", "value"); ①
        Map<String, String> map2 = Map.of("key", "value");
        Assertions.assertEquals(map2, map1); ①
        var anonymousSubclass = new Object() {
            final String name = "Peanut the Poodle";

            int age = 7;

        };
②
        Assertions.assertEquals(7, anonymousSubclass.age);
        Assertions.assertEquals("Peanut the Poodle", anonymousSubclass.name);
    }

}
```

① both variable definitions contain a type of `Customer`. The compiler knows it, and you know it because the right side of the expression makes it clear. So, all things being equal, why not use the more concise version?

② Indeed, in this case, the compiler knows *more* about the type than it would before, particularly in the case of anonymous subclasses. Suppose you need a throwaway object in which to stash some data temporarily? If you created an anonymous subclass of `Object` and assigned it to a variable of type `Object`, there'd be no way to dereference fields defined on the anonymous subclass. Traditionally, there was no way to reify *anonymous* subclass types because they were *anonymous*. But with `var`, you don't need to account for the type; you can let the compiler infer it.

This last bit - reifying anonymous subclasses comes in particularly handy when you're working with Java 8 streams abstractions and want to avoid having a host of throwaway classes created as side effects to conduct your data across the transformations.

The new `var` keyword can come in handy in a whole host of other smaller scenarios. You can use the `var` keyword for parameters in a lambda definition. It doesn't buy you much over just omitting `var` (or the type itself), except that you can now decorate the parameter with annotations. Neat!

Lambdas are the one fly in the proverbial ointment, however. Java doesn't have structural lambdas

like Scala, Kotlin, and other languages. Instead, you must assign the lambda to an instance of a functional interface like `java.util.function.Consumer<T>`. If the literal lambda syntax doesn't clarify what type that is, then the variable type definition itself must. So you can use `var` for every variable definition *except* lambdas. It's so dissatisfying! The agony of having a column of nice, clean `var's punctuated occasionally with standard variable definitions just because those variables happen to be lambdas! There is a way around it with casts, but I admit it isn't much better. Let's look at some examples.

```
package com.example.java.typeinference;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.HashSet;
import java.util.List;

class LambdasAndTypeInferenceTest {

    private String delegate(String s, Integer two) {
        return "Hello " + s + ":" + two;
    }

    @Test
    void lambdas() {
        MyHandler defaultHandler = this::delegate; ①
        var withVar = new MyHandler() { ②

            @Override
            public String handle(String one, int two) {
                return delegate(one, two);
            }
        };
        var withCast = (MyHandler) this::delegate; ③
        var string = "hello";
        var integer = 2;
        var set = new HashSet<>( //
                List.of(withCast.handle(string, integer), //
                        withVar.handle(string, integer), //
                        defaultHandler.handle(string, integer)));
        Assertions.assertEquals(1, set.size(), "the 3 entries should all be the same,
and thus deduplicated out");
    }

    @FunctionalInterface
    interface MyHandler {

        String handle(String one, int two);

    }
```

```
}
```

① I can't use `var` here because the compiler doesn't know to which functional interface type the method reference, `delegate(String, Integer)`, should be assigned

② I can use `var` here, but I've lost all the brevity of lambdas!

③ the only way I know around it is to cast the type like this. Ugh.

I tend to use the cast form a lot. Your sensibilities may vary, and given how new this feature is, I wouldn't be surprised if my sensibilities change in the future with respect to this feature, as well.

Enhanced Switch Expressions

Java has a new *enhanced* switch, which forms the basis of the gradual addition of *pattern amtching* to the Java language. It simplifies things considerably:

```
package com.example.java.switches;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

class TraditionalSwitchExpressionTest {

    @Test
    void switchExpression() {
        Assertions.assertEquals(respondToEmotionalState(Emotion.HAPPY), "that's
wonderful.");
        Assertions.assertEquals(respondToEmotionalState(Emotion.SAD), "I'm so sorry to
hear that.");
    }

    public String respondToEmotionalState(Emotion emotion) {
        var response = ""; ②
        switch (emotion) {
            case HAPPY:
                response = "that's wonderful.";
                break; ③
            case SAD:
                response = "I'm so sorry to hear that.";
                break;
        }

        return response;
    }

    enum Emotion {

①
        HAPPY, SAD
```

```
    }

}
```

① `Emotion` is an `enum`. There are only two possible values (for this simple example that doesn't at all reflect the rich gradient of human emotions) for a variable of type `Emotion`: `HAPPY` and `SAD`. We have branches for all known states in this' switch' statement. The compiler is satisfied that we have covered every possible value, so it doesn't insist on a `default` branch to handle any unforeseen values. There are no unforeseen values. We say that we've *exhausted* the range of values.

② we define a variable to which we assign the results of our processing.

③ take care to `break` for each branch. Otherwise, the execution flow will drop down to other branches, producing undesirable side effects.

The first example is a classic `switch` statement. There's nothing wrong, *per se*, but it's tedious, and I tend to avoid writing them.

```
package com.example.java.switches;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

class EnhancedSwitchExpressionTest {

    @Test
    void switchExpression() {
        Assertions.assertEquals("that's wonderful.", respondToEmotionalState(Emotion
.HAPPY));
        Assertions.assertEquals("I'm so sorry to hear that.", respondToEmotionalState
(Emotion.SAD));
    }

    private String respondToEmotionalState(Emotion emotion) { ①
        return switch (emotion) {
            case HAPPY -> "that's wonderful.";
            case SAD -> "I'm so sorry to hear that.";
        };
    }

    enum Emotion {

        HAPPY, SAD;

    }

}
```

① there's no intermediate variable! Each branch produces a value, and that value is the result of the `switch` expression and can be assigned to a variable or, as I do here, returned in one fell

swoop from the method as any other expression would.

Multiline Strings

This one is a super-small feature that punches well above its weight: Java supports multiline `String`s: hooray! There are a million opportunities for multiline `String`s: SQL queries, HTML, Markdown, Asciidoctor, Velocity templating (such as for emails), unit testing, etc.

```
package com.example.java;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.time.Instant;

class MultilineStringsTest {

    private final String instant = Instant.now().toString();

①
    private final String multilines = String.format("""
            <html>
            <body>
            <h1> Hello, world, @ %s!</h1> </body>
            </html>
            """, instant).trim();

②
    private final String concatenated = "<html>\n<body>\n" + "<h1> Hello, world, @ " +
instant + "!</h1> </body>\n"
            + "</html>";

    @Test
    void stringTheory() {
        Assertions.assertEquals(this.multilines, this.concatenated);
    }

}
```

① The first example uses a multiline `String` to represent some HTML markup

② The second variable recreates the same HTML markup, down to the padding and the newlines, but uses string concatenation and manually encodes newlines, as we used to have to do.

In the example, both variables `multilines`, and `concatenated`, are identical, but I the multiline `String` is much easier to wrangle.

Short and simple.

Records

Records are perhaps my second favorite new feature in Java. If you've ever used case classes in Scala, or data classes in Kotlin, you'll feel *almost* right at home.

Java introduced records, which are a new kind of type. Records, like enums, are a restricted form of a class. As a result, they're ideal for "plain data carriers," classes containing data not meant to be altered and only the most fundamental methods such as constructors and accessors. We'll use (and sometimes abuse) them *all* the time in this book. They're wonderful time savers. Need to model a read-only entity in your database that has accessors for all the constituent fields, a constructor, a functional `toString` implementation, a valid `equals` implementation, and a valid `hashCode` implementation? You'd better get codin'! That'll take a while. Or, you could use something like Lombok to code-generate all of that for you based on the presence of a handful of annotations. Or, you could use records.

Here's a trivial example. Suppose you want to return information about `Customer` entities *and* their associated `Order` data. Unfortunately, Java doesn't support multiple return types or provide suitable tuple types, so you need to create something to hold both types. Records to the rescue!

```
package com.example.java.records;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.List;

class SimpleRecordsTest {

    @Test
    void records() {
        var customer = new Customer(253, "Tammie");
        var order1 = new Order(2232, 74.023);
        var order2 = new Order(9593, 23.44);
        var cos = new CustomerOrders(customer, List.of(order1, order2));
        Assertions.assertEquals(2232, order1.id());
        Assertions.assertEquals(74.023, order1.total());
        Assertions.assertEquals("Tammie", customer.name());
        Assertions.assertEquals(2, cos.orders().size());
        IO.println("order components  " + order1.id() + ':' + order1.total()); ②
    }

①
    record Customer(Integer id, String name) {
    }

    record Order(Integer id, double total) {
    }

    record CustomerOrders(Customer customer, List<Order> orders) {
    }
```

```
}
```

① these three `record` definitions define three new types, each with accessors, constructors, and internal storage. Scarcely more than three lines of code for three brand new types!

② records also automatically expose accessor methods for the fields defined in the constructor. For example, if you want to read the `name` field of the `Customer` type, use `Customer#name()`. If you wish to read the `orders` field of the `CustomerOrders` type, use `orders()`. It took a while to accept that they they're in the form `x()`, and not in the form `getX()`, but I've come to love it. Mercifully, all the interesting libraries that need to know about this convention - JSON serializers, for example - already work well with it.

Records make perfect sense for immutable, data-centric types. They alleviate a whole host of boilerplate code. In the first edition of this book, I used the Lombok project [https://projectlombok.org] (which is brilliant) to synthesize the getters, setters, no-args, and all-arg constructors with just a few annotations. It worked, but it was still a handful of lines instead of the one-liners enabled by Java records. I love records!

I still occasionally use Lombok for other things, but it's nice to reduce my reliance on it further.

More controversially, I also sometimes use records to implement services and components quickly. After all, you can have methods on a record. Of course, record implementations can't extend classes, but one doesn't need to do that a lot. There is the undesirable side-effect of having accessor-methods that expose the state - `dataSource()` or whatever - but, for whatever reason, I don't care. It doesn't cost me anything when I use it. If my code grows large enough to need hierarchies or interface implementations, I'll change it. If the code grows enough to worry about the leaking state, I'll change it. But the immediate, short-term effect of having more concise, approachable, readable code seems to make sense to me. Maybe one day that'll change?

Behind the scenes, a record creates a default constructor whose parameters match the types and rarity of the record header. Records can have other constructors, but they need to delegate to the default constructor.

```
package com.example.java.records;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;
import org.springframework.util.Assert;
import org.springframework.util.StringUtils;

class RecordConstructorsTest {

    @Test
    void multipleConstructors() {
        var customer1 = new Customer("test@email.com");
        var customer2 = new Customer(2, "test2@gmail.com");
        Assertions.assertEquals(customer1.id(), -1);
        Assertions.assertEquals(customer2.id(), 2);
    }
```

```
    record Customer(Integer id, String email) { ①

        Customer { ②
            Assert.notNull(id, () -> "the id must never be null!");
            Assert.isTrue(StringUtils.hasText(email), () -> "the email is invalid");
        }

        Customer(String email) {
            this(-1, email);
        }
    }

}
```

① the default constructor is the one defined in the record header

② If you want to act on the fields passed in the default constructor, create a constructor with no parameters and do the work there. This no-parameter constructor and the record header taken together are the default constructor. If you want to have an alternative constructor, you can do that so long as you forward to the default constructor. I use the default constructor to initialize the `id` to `-1`. This way, it's impossible to initialize the record with a `null` `id` field.

Sealed Types

Sealed types are a novel feature that hasn't reached its full potential yet. The basic idea is to constrain the number of subtypes for a given type. Why would you do this? Well, it's all a bit *exhausting*! Or should I say, it's all about exhausting the extent to which the runtime needs to support virtual (polymorphic) dispatch?

Sound complicated? It's not really. Ever have a method in a parent type that the child type overrides? The ability to call that method on an instance of that child type, in terms of the parent type's interface, is called polymorphism.

Polymorphic, or *virtual*, dispatch requires a lookup in the virtual function table, which in theory takes time. The runtime wouldn't have to do that if it could say conclusively that a given type can never be subclassed, such as with a final type. The `final` keyword is a bit of a sledgehammer, however. It forecloses entirely on the possibility of on any kind whatsoever subclassing the final type. You might have a hierarchy but wish to keep it shallow, and well-known. Alternatively, you could make everything package-private (simply omit `public` modifier on the class), which means that only types in the same class could subclass the type. This would probably work, but of course, somebody else could come along and create types in the same package in their `.jar`. There's traditionally not been a lot of great ways to keep a hierarchy shallow until now. Sealed types can help. They let you constrain the number of subclasses to a known set.

Let's look at an example that is building out a

```
package com.example.java;

import org.junit.jupiter.api.Assertions;
```

```
import org.junit.jupiter.api.Test;
import org.springframework.util.Assert;

class SealedTypesTest {

④
    private String describeShape(Shape shape) {
        Assert.notNull(shape, () -> "the shape should never be null!");
        if (shape instanceof Oval)
            return "round";
        if (shape instanceof Polygon)
            return "straight";
        throw new RuntimeException("we should never get to this point!");
    }

    @Test
    void disjointedUnionTypes() {
        Assertions.assertEquals(describeShape(new Oval()), "round");
        Assertions.assertEquals(describeShape(new Polygon()), "straight");
    }

①
    sealed interface Shape permits Oval, Polygon {

    }

②
    static sealed class Oval implements Shape permits Circle {

    }

③
    // static final class Rhombus implements Shape {}

    static final class Circle extends Oval {

    }

    static final class Polygon implements Shape {

    }

}
```

① we'll start with a `Shape` and permit two direct subclasses, `Oval` and `Polygon`.

② a subclass of a sealed type must be `final` or `sealed` and explicitly name its subclasses. A sealed type's subclasses may themselves be sealed types, permitting further subtypes.

③ the following declaration does not compile, as it is not one of the explicitly permitted subclasses

④ the `describeShape` method is written so that we can exhaustively handle every subclass.

Sealed types help the compiler, too. The compiler can exhaustively determine every case of every possible subtype, which has implications for the future, as Java looks to better incorporate simple pattern matching into Java. Imagine the possibilities here, and you can kind of see how sealed types might play with the new `switch` expressions, too.

Right now, I don't recommend using `sealed` types. I tend to think types should be open by default. You just don't know what scenario will arise in the future that changes your fundamental assumptions. Furthermore, sealed subtypes are `final`, inhibiting tools like Spring and Hibernate, which must subclass your types to proxy them.

Smart Casts

This feature is one of those things that I don't use all that often in application code, but it's pretty useful when writing infrastructure code when dealing with polymorphism. Essentially, the feature spares you from having to create an intermediate variable and casting after you've already done an `instanceof` check.

```
package com.example.java;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.HashSet;

public class SmartCastsTest {

    @Test
    void casts() {
        interface Animal {

            String speak();

        }

        class Cat implements Animal {

            @Override
            public String speak() {
                return "meow!";
            }

        }

        class Dog implements Animal {

            @Override
            public String speak() {
                return "woof!";
            }
```

```
        }

        var newPet = Math.random() < .5 ? new Cat() : new Dog();
        var messages = new HashSet<String>();

①
        if (newPet instanceof Cat) {
            var cat = (Cat) newPet;
            messages.add("the cat says " + cat.speak());
        }
②
        if (newPet instanceof Cat cat) {
            messages.add("the cat says " + cat.speak());
        }

        if (newPet instanceof Dog dog) {
            messages.add("the dog says " + dog.speak());
        }

        Assertions.assertEquals(messages.size(), 1);
        Assertions.assertTrue(messages.contains("the dog says woof!") || messages
.contains("the cat says meow!"));

    }

}
```

① this is a classic example, where we determine the subtype and then create a variable cast to the appropriate type. If ever we change the type definition, we have to replace it both in the cast and in the `instanceof` check.

② the more excellent, newer alternative uses a smart-cast, sparing us the extra variable and cast.

This feature looks almost exactly like a similar feature in Kotlin, and I love it.

Function Interfaces, Lambdas and Method References

Java 8 introduced lambdas. They let us treat functions as first-class citizens that can be assigned and passed around. Well, almost. In Java, lambdas are slightly limited; they're *not* first-class citizens. But, close enough! They *must* have return values and input parameters compatible with the method signature of the sole abstract method of an interface. This interface, one with a single abstract method intended for use as a lambda, is called a *functional interface*. (Interfaces may also have *default*, non-abstract methods where the interface provides an implementation.)

There are some very convenient and oft-used functional interfaces in the JDK itself. Here are some of my favorites.

- `java.util.function.Function<I, O>`: an instance of this interface accepts a type `I` and returns a type ` O'. This is useful as a general-purpose function. Every other function could, in theory, be generalized from this. Thankfully, there's no need for that as several other handy functional interfaces exist.

- `java.util.function.Predicate<T>`: an instance of this interface accepts a type `T` and returns a `boolean`. This is an ideal filter when you're trying to sift through a stream of values.
- `java.util.function.Supplier<T>`: A supplier accepts no input and returns an output.
- `java.util.function.Consumer<T>`: A consumer is the mirror image of a `Supplier<T>`; it takes an input and returns no output.

You can also create and use your custom functional interfaces. Let's look at some examples.

```
package com.example.java;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.function.Function;

class LambdasTest {

    @Test
    void lambdas() {
        Function<String, Integer> stringIntegerFunction = str -> 2;①
        interface MyHandler {

            String handle(String one, Integer two);

        }
        MyHandler withExplicit = (one, two) -> one + ':' + two;②
        Assertions.assertEquals(stringIntegerFunction.apply(""), 2);
        Assertions.assertEquals(withExplicit.handle("one", 2), "one:2");
        var withVar = (MyHandler) (one, two) -> one + ':' + two;③
        Assertions.assertEquals(withVar.handle("one", 2), "one:2");
        MyHandler delegate = this::doHandle; ④
        Assertions.assertEquals(delegate.handle("one", 2), "one:2");

    }

    private String doHandle(String one, Integer two) {
        return one + ':' + two;
    }

}
```

① You can use existing functional interfaces in the JDK…

② you can define your own interfaces and use them as functional interfaces

③ You may use `var` and lambdas together with this (admittedly unsightly) cast.

④ and if you have existing methods whose return types and input parameters line up with the single abstract method of a functional interface, then you can create a method reference and assign it to an instance of that type.

Pattern Matching

We've seen the smart casting `if` statement, and we've seen the switch expression. At the heart of both of these new things is *pattern matching*, wherein we match a value and then extract the value for use. We've only really seen that it's possible to easily match values thus far, so let's revisit that a bit.

```
package com.example.java.patternmatching;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertTrue;

class PatternMatchingTest {

①
    final String authenticateMessage = """
                please authenticate, so that we can
                get to know you a little better.
            """.stripIndent().stripLeading().stripTrailing();

    final String customerMessage = "the customer's name is %s";

②
    String showMessageWithIf(BrowserClient browserClient) {
        var message = "";
        if (browserClient instanceof Customer(var id, var name)) {
            message = this.customerMessage.formatted(name);
        }
        else if (browserClient instanceof UnknownUser) {
            message = this.authenticateMessage;
        }
        return message;
    }

③
    String showMessageWithSwitch(BrowserClient browserClient) {
        return switch (browserClient) {
            case Customer(var id, var name) -> this.customerMessage.formatted(name);
            case UnknownUser() -> this.authenticateMessage;
        };
    }

    @Test
    void patternMatching() throws Exception {
        assertTrue(showMessageWithIf(new Customer(1, "Josh")).contains(this
.customerMessage.formatted("Josh")));
        assertTrue(showMessageWithSwitch(new UnknownUser()).contains(this
.authenticateMessage));
    }
```

```
④
    sealed interface BrowserClient permits UnknownUser, Customer {

    }

    record UnknownUser() implements BrowserClient {
    }

    record Customer(Integer id, String name) implements BrowserClient {
    }

}
```

① for our examples, we're going to work with a shallow hierarchy rooted in a type called `BrowserClient`. Imagine that when an HTTP request comes in, we adapt the request into one of either a known `Customer` or an `UnknownUser`.

② we'll do some analaysis of the incoming user type and display a message as appropriate. Since we're going to prepare these same messages two different ways, I've extracted them out into separate environment variables.

③ let's use the smart `if` statement to deduce which message to present. Note that we're now switching on an instance of `BrowserClient`. In the first line we do two things at once: we check that the variable is an instance of `Customer` and if so, we hoist out - we *extract* not just the `Customer` variable, but instead the constituent component fields of the record itself - `id`, and `name` - into their own new variables. Super concise. In the second test, we don't need a pointer to the `UnknownUser` or its field, so we do nothing with it save match it.

④ the if / else structure worked fine but it meant that we had to stash a variable and then make sure that only one branch of the test was ever evaluated. The structure as written also gives us nothing in the way of comfort. What happens when we add a new type to the hierarchy? We need to handle that branch, but the compiler will let us ignore it. Let's use our new friend the `switch` expression. The switch expression has access to thes ame pattern matching super powers as the `if`, but it does us two new favors. The compiler wwill complain if we add a new type to the hierarchy because it knows that the types are `sealed` and therefore there is an exhaustive set of types and our `switch` hasn't defined a `default` branch. Thanks, compiler! And, to make things jsut that much cleaner, the `switch` is an expression, so we can assign the results of the `switch` to a variable or just return the result of the expression directly.

I bet you never thought you'd see the day that the `switch` was more elegant than an `if`, did you?

Java language architect Brian Goetz talks about the combination of some of the features that we have just covered - pattern matching, sealed types, the smart switch expression - as *data oriented programming*. Java has reigned supreme in large monolithic codebases, he explains, because it provides good object-oriented support and strong enforcement of privacy, security and encapsulation. In such a codebase, the boundary on which different parts of the program align is typically an object interface, which the magic of polymorphism incentivises. But programs aren't usually large, sprawling, deeply rooted graphs of objects these days. More often than not they're smaller programs dealing with ad-hoc messages coming in over the wire and being handled in terms of dumb, but typed, carriers. There's no hierarchy at all! In this context, Java 8 was starting to

feel mighty clunky. But no more. With Java 25, it's trivial to define small ad-hoc objects and to vary behavior based on those objects. In a sense, Java 25 introduces a new kind of dispatch: not the dynamic dispatch of yore, but a dispatch for small ad-hoc objects.

And just like that, without us ever realizing it, Java 25 manages to introduce support for a completely new style of programming: data oriented programming. And I bet you were only just getting used to the possibilities of the rudimentary functional programming support in Java 8!

Java 25, Project Loom, and Virtual Threads

Project Loom brings transparent fibers to the JVM. As things stand today, in Java 20 or earlier, IO is blocking. Call `int InputStream#read()`, and you might have to wait for the next byte to finally arrive. In `java.io.File`-centric IO, there's very rarely much of a delay. On the network, however, you just can't really ever know. Clients might disconnect. Clients be people, driving through a tunnel and getting spotty signal coverage. During this time, the program flow is said to be *blocked* from proceeding on the thread of execution. In the following snippet, we have no way of knowing when we'll see the word "after" printed. Might be a nanosecond from now. Might be a week from now. It's *blocking*.

```
Executor executor = Executors.newCachedThreadPool();
executor.submit(() -> {
    InputStream in = ...
    IO.println("before");
    int next = in.read();
    IO.println("after");
});
```

This means that we can't address any other requests with this thread until it's finished doing whatever it was doing. We'll need more threads.

This is bad enough, but it's made worse by the architecture of threading in Java prior to Java 25 where each thread maps, more or less, to a native operating system thread. It's expensive to create more threads, too, taking about two megabytes of RAM. We're going to need a *lot* of threads to handle requests if our existing threads are blocked long enough. Many, many, times more threads. Or, we're going to need to find a way to avoid *blocking* on the precious few threads we do have.

We could use non-blocking IO, as enabled by Java NIO (java.nio.*). In this model, you ask for bytes and register a callback that the runtime executes only when there are actually bytes available. No waiting, no blocking. This approach has the significant benefit of keeping us off threads when there's nothing to be done, allowing others to use those threads in the meantime. It's a bit tedious, however, and low-level. Spring has amazing support for reactive programming, which offers a functional-style programming model on top of non-blocking IO. It works well. But, it requires changing the way you write code. If you're not used to it, it can be a bit daunting, too. Wouldn't it be nice if you could just take that existing code, as demonstrated above, and have it do the right thing, transparently moving the flow of execution off the thread when there's nothing happening, and then resuming the flow of execution when there is? This way, we could keep our relatively simpler code, and get the benefits of non-blocking IO. Absolutely it would. And now you can. Project Loom is straightforward at the high levels: if you run your code in a *virtual thread* (just another type of

`java.lang.Thread`), then the runtime will detect *blocking* operations like `InputStream#read()`, `Thread.sleep(long)`, etc., and automatically move the code off the thread its on if it's in a state where nothing is happening. It'll move the code back on to the thread once there's something to be, once the blocking operation has stopped blocking. In the case of reading, that means it'll resume once there are bytes to read. In the case of `Thread.sleep`, it'll resume once the sleep time has elapsed. And you don't have to do anything to get it to work. Let's look at an example that I shamelessly stole from Oracle Java advocate José Paumard.

```
package com.example.java.loom;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;
import java.util.concurrent.ConcurrentSkipListSet;
import java.util.stream.IntStream;

class LoomTest {

    private static Set<String> observe(int index) {
        var before = Thread.currentThread().toString();
        try {
            Thread.sleep(100);
        } //
        catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        var after = Thread.currentThread().toString();
        return index == 0 ? new HashSet<>(Arrays.asList(before, after)) : Set.of();
    }

    @Test
    void threads() throws Exception {
        var switches = 5;
        var observed = new ConcurrentSkipListSet<String>();
        var threads = IntStream.range(0, 1000)①
            .mapToObj(index -> Thread.ofVirtual()②
                .unstarted(() -> {
                    for (var i = 0; i < switches; i++) ③
                        observed.addAll(observe(index));
                }))
            .toList();
        for (var t : threads)
            t.start();
        for (var t : threads)
            t.join();
        Assertions.assertTrue(observed.size() > 1); ④
    }
```

```
}
```

① This program launches a 1000 virtual threads. This is an arbitrary number, but it's also enough that it'll create contention - a need to share the precious few resources available to the machine.

② the program sleeps five times in each thread, for about 100 milliseconds each. This is not a long time, but it's enough that it forces the runtime to have to interleave, to *share*, the actual operating system threads.

③ In each thread we call the `observe` function, which notes the current thread, sleeps, and then notes the current thread again. We don't want all the threads, however, as that would be too many. So we sample: if we're on the first thread (whose index would be `0`) launched, not the information; otherwise: do nothing with the information and move on.

④ by the end of the run, we've captured the thread on which our first of a thousand threads executed and stored them in a `Set<String>`, where the values will be deduplicated. Interestingly, there is *probably* more than one value in the set! Remember, we only noted the thread of execution for *one* thread - the *same* thread! - and somehow - without us asking or doing anything, it got moved around. Print the values out, and you'll see the code ran on the same *virtual* thread, but on multiple carrier threads.

The result is that while we were `Thread.sleep`-ing, the carrier thread on which we were executing was made available to whatever else required it in the JVM. This efficient time-sharing means that now we can handle a *lot* more requests with the same codebase. All we had to do was make sure to run the code on a *virtual* thread.

You don't have to use the `Thread.ofVirtual()` construction, either. There's an `Executor` implementation, too: `Executors.newVirtualThreadPerTaskExecutor()`. It's not a pool, however. Threads are no longer expensive, after all.

That's easy enough. In a typical Spring Boot application, however, there are countless places where you'll need to override the default configured `Executor`, but in Spring Boot 3.2 we can do that for you. Specify `spring.threads.virtual.enabled=true` and you're all set.

Loom is especially important in the context of the Spring Authorization Server, because it uses traditional blocking IO. If you want to easily scale it up, consider using virtual threads and Java 25 when running the Spring Authorization Server. Who knows, you may not need to add new instances. You'll get to have your cake and eat it too: scalable, but simpler, code.

Next Steps

I didn't intend for this chapter to be a thorough introduction to all things Java. Just an amuse-bouche for all the cool stuff you might see me do in my code independent of Spring. Java 25 is a very compelling way to write code, and it's no coincidence that it's starting to look more and more like languages like Kotlin, which is also a lovely way to write Spring Boot-based applications. Hopefully, this chapter has been persuasive, and you're ready to turn the page. :code: ..

From Beans to Boot

You know, I am assuming that by the time you're reading this that you have some background with Spring. I assumed the same thing about one of earlier books, *Reactive Spring*, too, but I included a *very* simple discussion of some of the core principles of Spring in that book and - to my utter shock and chagrin - it was one of the most loved aspects of the book! I couldn't believe it!

So, here we go. We're going to do the same sort of thing, but of course greatly udpated to reflect the amazing moemnt in which we fnd ourselves.

when spring first came out we talked about the "spring triangle" the three pillars that made the Spring Framework so useful for people: * portable service abstractions * dependency injection * autoconfiguration

more recently, with the introduction mof Spring Boot, we've added a new dimension of utility to the framework: autoconfiguration. So I guess its more like a "Spring square" now, come to think of it.

At the core of Spring as a framework is the fact htat it needs to understand how your objects are wired together. But why? why bother telling it these things? Why not just wire things yourself?

at the microlevel, this isnt an unreasonable quesiton. spring is just java and i can write java without spring, so why bother with Spring?

let's introduce a *trivial* example to enforce the idea. Were going to read `Dog` data from a SQL database, PostgreSQl, running on our machine. Well use the follwoing Docker Compose file to stand up the instance of our database.

services: postgres: image: 'postgres:latest' environment: - 'POSTGRES_DB=mydatabase' - 'POSTGRES_PASSWORD=secret' - 'POSTGRES_USER=myuser' ports: - '5432:5432'

- spring 'square' : psb, aop, di, autoconfig
- basic repository
- core java
- DataSource inline
- use DI to extract it out; nothing fancy just polymorphism.
- use JdbcClient
- what about tx?
- oop says dry; extract single responsibility to another class
- TransactionalDogRepository is a composite that wraps the target one
- platformtxmanager
- transactiontemplate
- still very tedious tho is there some way to programatically generate new implementations of the interface?
- jdk proxies

- now we can wrap our target with our new `transactional` method
- its nie but things are getting out of hand. that said, our code is approachable enough for now. bear in mind were (barely) doing any persistence
- lets see if spring can improve things
- turn everything into beans
- there is a lifecycle at play. i can control the beans construction and destuction. they can implement InitializingBean && DisposablBean
- these callbacks are nbut one way to participate in important lifecycle events. another is to use `ApplicationEvent`s. there are a ton of them! lets implement a simple `@EventListener`
- the lifecycle is interesting. when the app starts up and shuts down ur code can hook into the lifecycle.
- > ingest → BeanDefintiions → beans
- BeanFactoryPostProcessor lets u see the BeanDefintions
- what we want is BeanPostProcessor
- extract the transactional support to this based on presence of marker interface `Tx`
- obviously, if u have used spring at all before then u prolly know where this is going: we've already done this exact magic trick. let's refactor to use `@transactional`. just need to enable it with `@EnableTransactionManagment`
- nice but of course its tedious to define beans with full config every time. lets use implicit configuration and `@ComponentScan`
- component scanning for our repository by adding `@Repository`
- stereotype annotations are neat. u can create ur own, too
- this is *implicit* configuration
- as opposed to the explicit configuration
- Spring Framework 7 introduced `BeanRegistrar`s, too, btw. here's how that would look.
- anyway weve done a lot here. but weve got harccoded configuration. let's extricate the values to properties (`application.properties`) and use the `Environment`
- it's nice! but theres a lot of boilerplate thatll prolly be the same from one appt o another.
- lets use Spring Boot. same `ApplicationContext` but the code is much simpler. We can delete everything except the DogRepository and the `test` method
- this works because of AutoConfiguration. you can create your own too. the autoconfig is activated by the presence of `starters`.
- and, being honest, im not sure we ever needed to implement the repository ourselves. lol. lets just use the Sprig Dats JDBC project.
- note that weve also now got good ascii art. we've got an actuator. weve got a web server.
- we can use virutal threads!
- remember the BEanDfinitions? we can turn this into a graalvm native image!
- use spring boot's build plugin to create a docker image, too